

Introduction to Legged Robots

Prof. Michele Focchi, Associate Professor
University of Trento



Outline

Brief History of legged Robots

Floating Base Robots

Basic Concepts in Legged Locomotion

Modeling Legged robots

Towards Newton-Euler equations

Quasi-static Trunk Controller

Brief history of legged robots



First computer-controller robot (Raibert, 70s – 80s)



OSU ASV
(Adaptive Suspension Vehicle), 1984



P2, Honda
(1996)

ZMP concept
(Vukobratovic, 1972)

1970

1980

1990

2000



First Humanoid Robot
(1973 - Waseda University, Japan)



The pre-robotic period
First walking machines were tele-operated by humans: GE Walking Truck (~1960s)

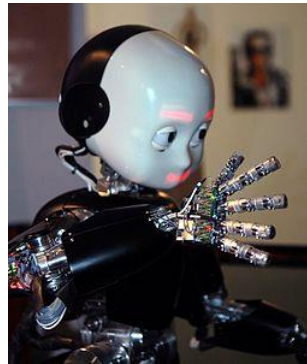


Purely passive dynamics
(McGeer, 1990)

Brief history of legged robots



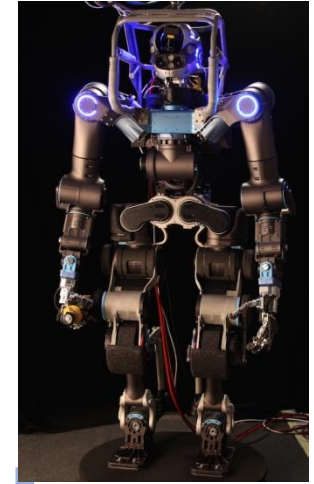
HRP-2
(Kawada, 2002)



iCub
(IIT, 2009)



Toro
(DLR, 2013)



Walkman
(IIT, 2015)

2000

2010

2020



ASIMO,
Honda (2000)



BigDog, Boston Dynamics (2008)



HyQ, IIT (2010)



ANYmal, ETH
(2016)



Atlas, Boston Dynamics (2019)



HyQReal, IIT (2019)

Brief history of legged robots



Spot, Boston Dynamics,
Industrial Inspection(2020)



Menzi Muck, Large Scale Legged
Locomotion and Manipulation
(2024)

2020

2021

2023

2024

2026



Go1, Unitree
(2021)



Digit, Agility
Robotics,
Warehouse
(2023)



G1, Unitree
(2024)



Optimus Gen 3, Tesla
(2026)

Darpa Robotics challenge (2012)

Walking is difficult...



AS2 robot (2026)

Unitree As2-W

Power Pioneer

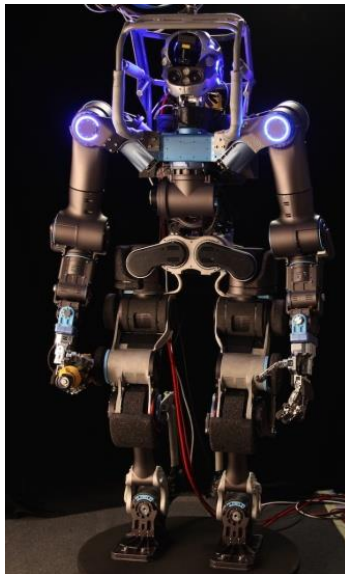
Compact yet Exceptional



Humanoids VS Quadrupeds

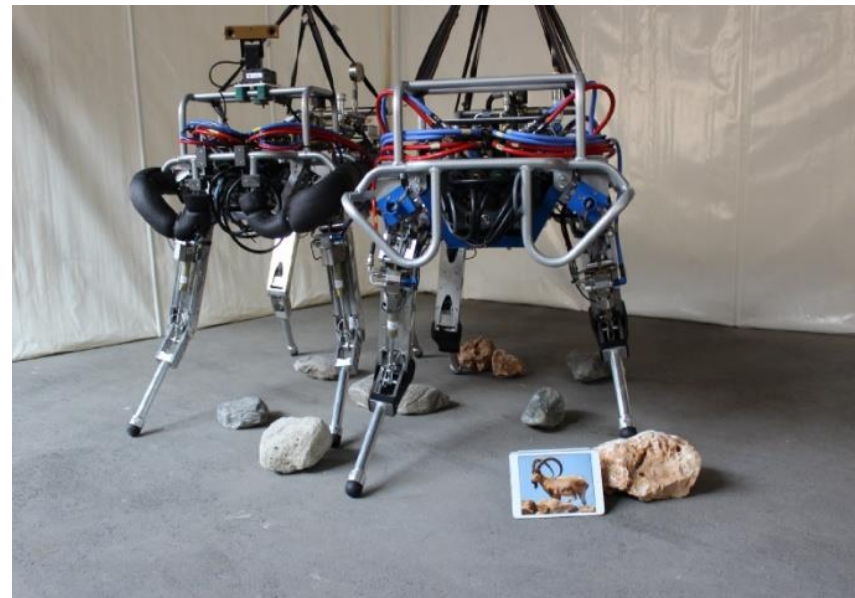
Why Humanoids?

- *adaptability*: humanoids can work in environments suitable for humans and use machines designed for humans
- *psychological and commercial reasons*: humanoids have a major appeal because of human-like appearance



Why quadrupeds?

- Higher stability due to the larger support polygon
- Smaller feet (point-like assumption)
- Different kind of gaits (walk, trot, pace...)



Outline

Brief History of legged Robots

Floating Base Robots

Basic Concepts in Legged Locomotion

Modeling Legged robots

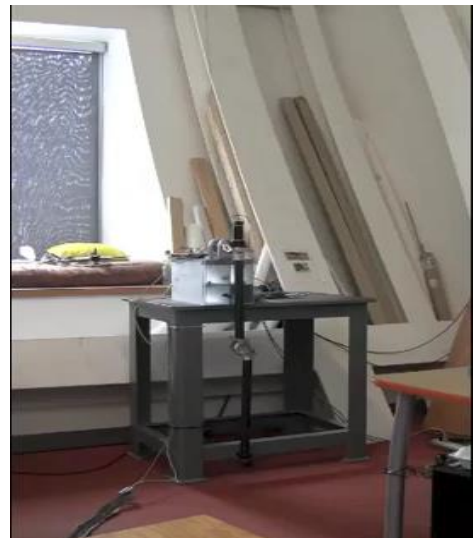
Towards Newton-Euler equations

Quasi-static Trunk Controller

Underactuated Robots

- Let's take a step back to understand where legged robots fit within the broader robotics landscape...
- **Definition of Underactuation:** more degrees of freedom than the number of actuators
- In general, underactuation is considered as a global property of the system: we will refer to a system as underactuated if it is underactuated in all states and times.
- Powerful robot arms on factory do have actuator limits, but we can typically design controllers for them as though they were fully actuated.
- **Controllability:** under-actuated systems cannot follow arbitrary trajectories, but that does not mean that they cannot arrive at arbitrary points in state space (e.g. using contact forces or taking into account the under-actuated dynamics)

Acrobot



Dynamics of Underactuated Robots

- Robot Dynamics:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$$

q : Generalized coordinates

$\mathbf{M}(\mathbf{q})$: Mass matrix

$c(q, \dot{q})$: Centrif. and Coriolis forces

$g(q)$: Gravity forces

τ : Generalized forces

- we can, without loss of generality, reorganize the joint coordinates as:

$$\mathbf{q} = \begin{bmatrix} \mathbf{q}_u \\ \mathbf{q}_a \end{bmatrix}$$

$\mathbf{q}_u \in \mathbb{R}^{n-m}$ Passive joints (underactuated)

$\mathbf{q}_a \in \mathbb{R}^m$ Actuated joints

- $\mathbf{S}$ is the selection matrix: it maps the control inputs to generalized forces performing work on $\mathbf{q}$:

$$\mathbf{S} = \begin{bmatrix} \mathbf{0} \\ \mathbf{I} \end{bmatrix}$$

- the system dynamics can be partitioned into the **underactuated** and the **actuated** parts:

$$\begin{bmatrix} \mathbf{M}_{uu}(\mathbf{q}) & \mathbf{M}_{au}^T(\mathbf{q}) \\ \mathbf{M}_{au}(\mathbf{q}) & \mathbf{M}_{aa}(\mathbf{q}) \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}}_u \\ \ddot{\mathbf{q}}_a \end{bmatrix} + \begin{bmatrix} \mathbf{c}_u(\mathbf{q}, \dot{\mathbf{q}}) \\ \mathbf{c}_a(\mathbf{q}, \dot{\mathbf{q}}) \end{bmatrix} + \begin{bmatrix} \mathbf{g}_u(\mathbf{q}, \dot{\mathbf{q}}) \\ \mathbf{g}_a(\mathbf{q}, \dot{\mathbf{q}}) \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \boldsymbol{\tau} \end{bmatrix} = \mathbf{S}\boldsymbol{\tau}$$

Example of underactuation: floating base robots

- Industrial fixed-base robots are fast and accurate in a limited, structured, known, static workspace
- Typically each joint is driven by 1 actuator (e.g., electric, hydraulic, pneumatic).
- To be useful in the outside world, robots must be able to move freely in large, **unstructured, uncertain**, dynamic environments



Floating base (or mobile) robots

- Floating base robots are a class of underactuated robot where the base link is **under-actuated**

Examples of floating base (mobile) robots

Underwater



Seagoo ROV
(inspection)

Space



A robotic arm on the International Space Station grabbed onto a cargo capsule from the SpaceX spacecraft. NASA, via Associated Press

NASA SpaceX
(space manipulation)

Air



Amazon Prime Air Quadrotor
(delivery)

Wheels



WooWee Rovio
(monitoring)

Tracks

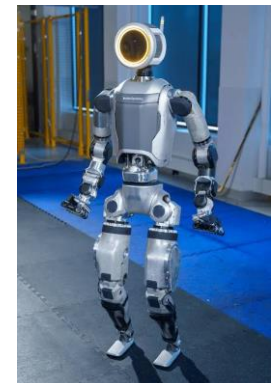


Indumil
(demining)



AS2 Quadruped
(inspection)

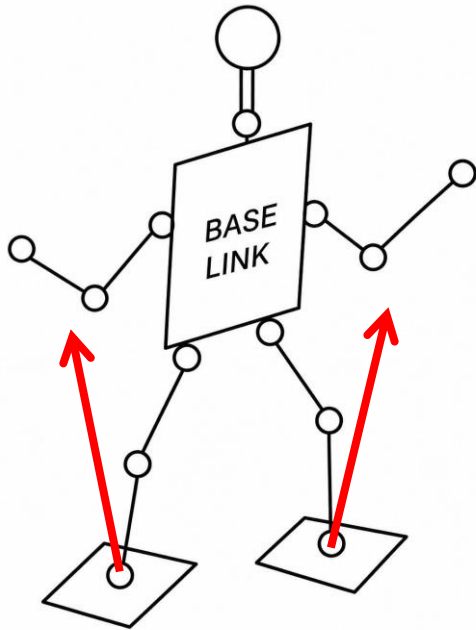
Legs



ATLAS Humanoid
(research?)

Legged robots

- Legged robots are a peculiar type of floating base robots with single chain kinematic structures (legs) that branch from the base (no kinematic loops)



- Differently from fixed based robot legged robots can fall...



- The base can be controlled through the action of contact forces**

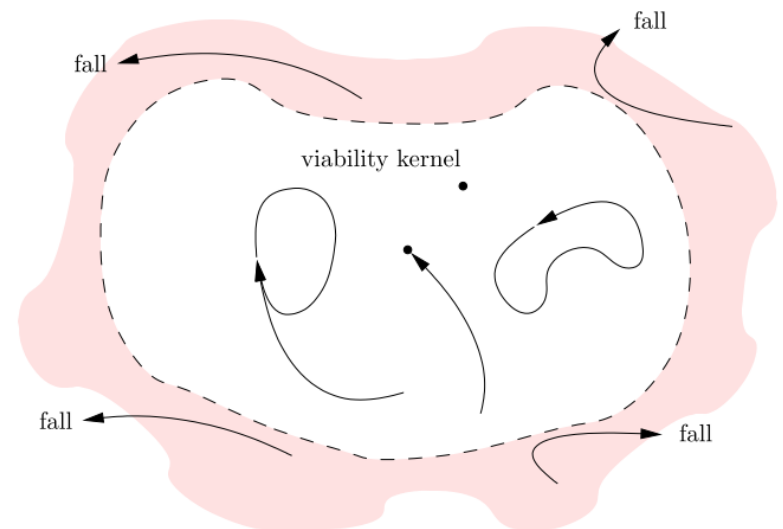
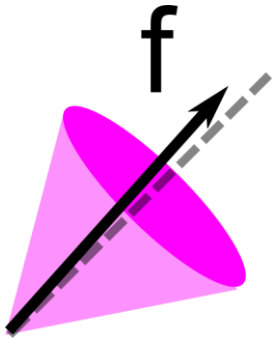
- Legged locomotion is about creating the right contact forces on the ground to keep balance and move in the desired direction

Scientific challenges in legged robots

C1

Heavily constrained

- Actuation limits
- Kinematic limits
- Friction constraints
 - Friction cones
 - Unilaterality ($f > 0$)



Scientific challenges in legged robots

C1

Heavily constrained

C2

Dynamics

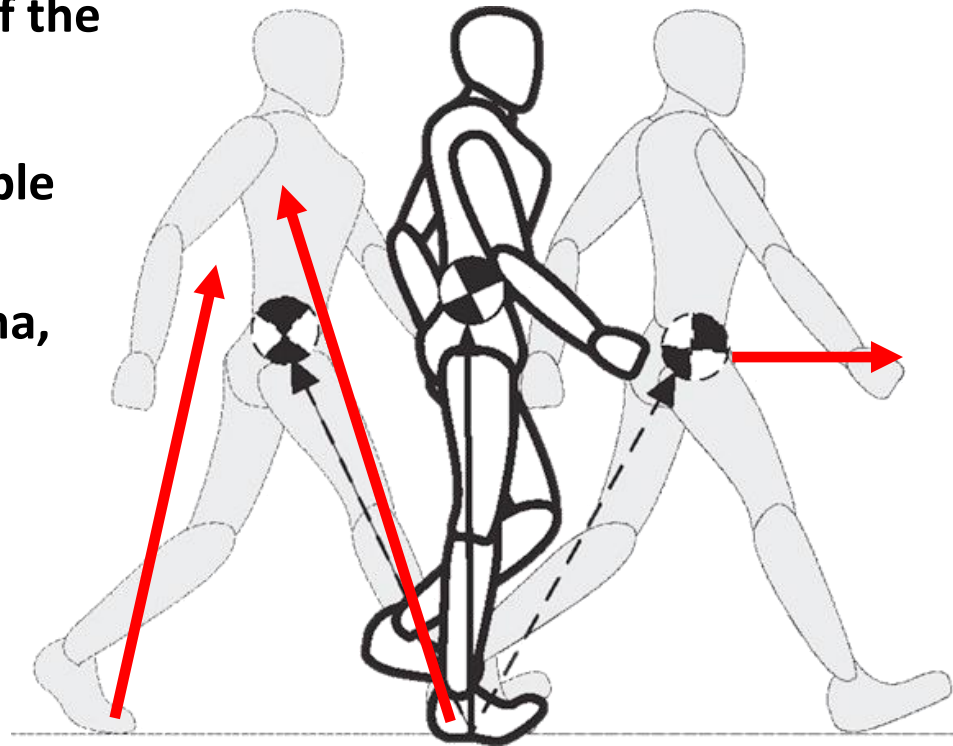
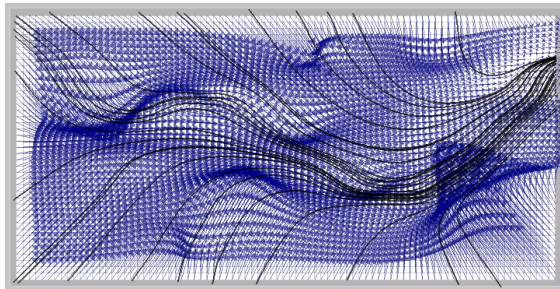
- Unstable: divergence of the Center of mass
- Hybrid: not differentiable
- Non-linear: local minima, initialization problems



optim



optim



Scientific challenges in legged robots

- “Temporarily” underactuated
- Forces can only be created when a foot is touching the environment.
- This restricts where the forces can act to, the surface of the terrain and nowhere else.
- In contrast drones, can always exert a force at the center of their rotors, independent of the terrain
- To generate a force at a different location, we must accept a duration during which no force and is possible to re-position the foot (i.e. leg does not significantly contribute to moving the base)



The difficulty to control legged systems comes largely from the various restrictions for the creation of external forces

Scientific challenges in legged robots

C1

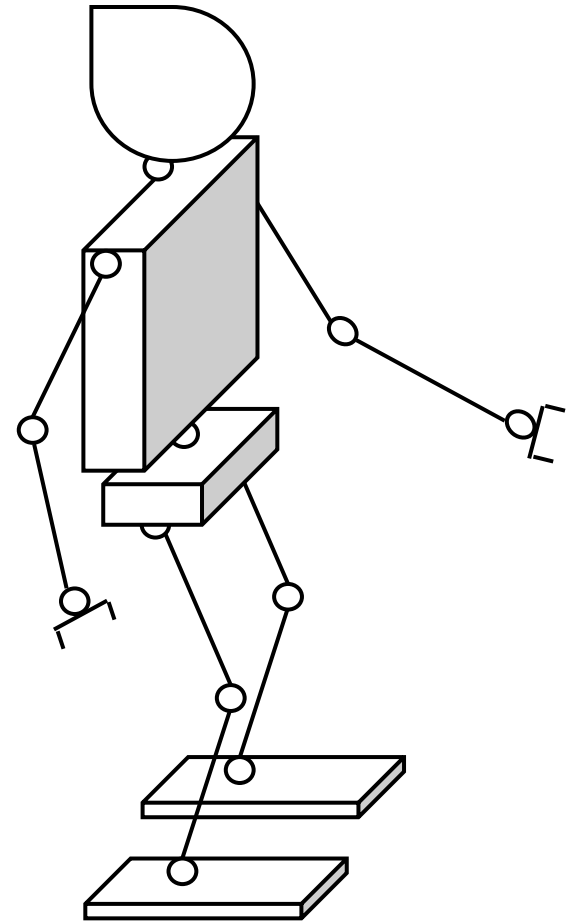
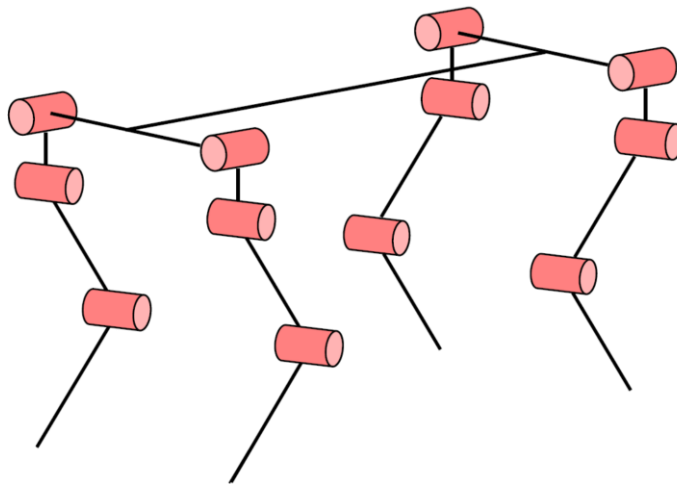
Heavily constrained

C2

Dynamics

C3

Many Degrees of Freedom



Scientific challenges in legged robots

C1

Heavily constrained

C2

Dynamics

C3

Many Degrees of Freedom

C4

Computation time

Full state: 24d

Prediction horizon: 1-2 s

Number of constraints: 40

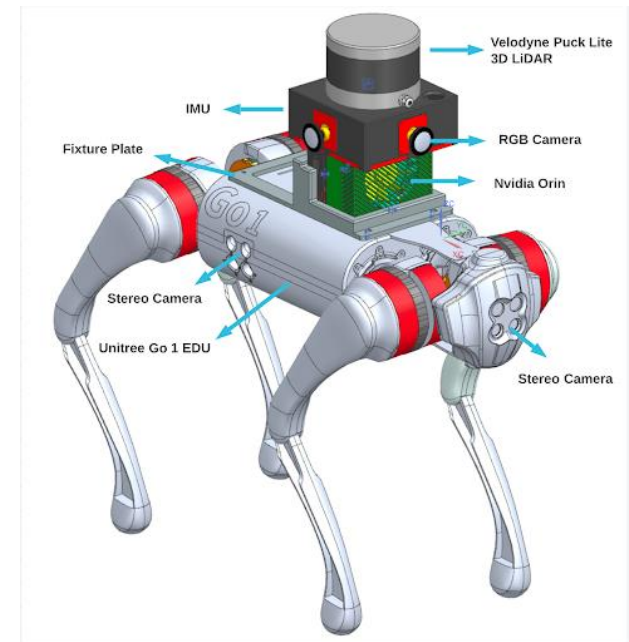
Computation time < 20ms



Scientific challenges in legged robots

C5 Lack of proprioception

- no (onboard) sensors that allow to directly measure the **base position** and **linear velocity**.
- use sensor fusion algorithms in order to estimate the pose from different other sensor information/motion capture
- A typical sensor suite for a robot includes cameras, LiDAR, encoders, and IMU



Outline

Brief History of legged Robots

Floating Base Robots

Basic Concepts in Legged Locomotion

Modeling Legged robots

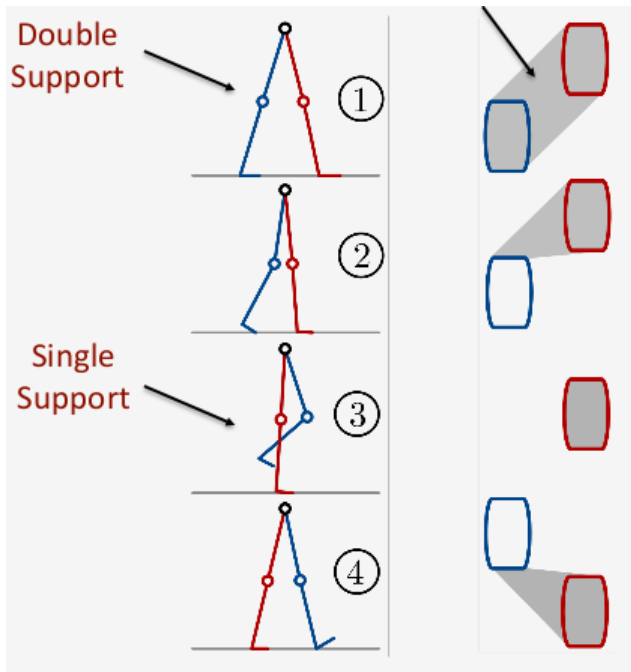
Towards Newton-Euler equations

Quasi-static Trunk Controller

Support Polygon (SP)

- Definition: Convex hull of the contact points

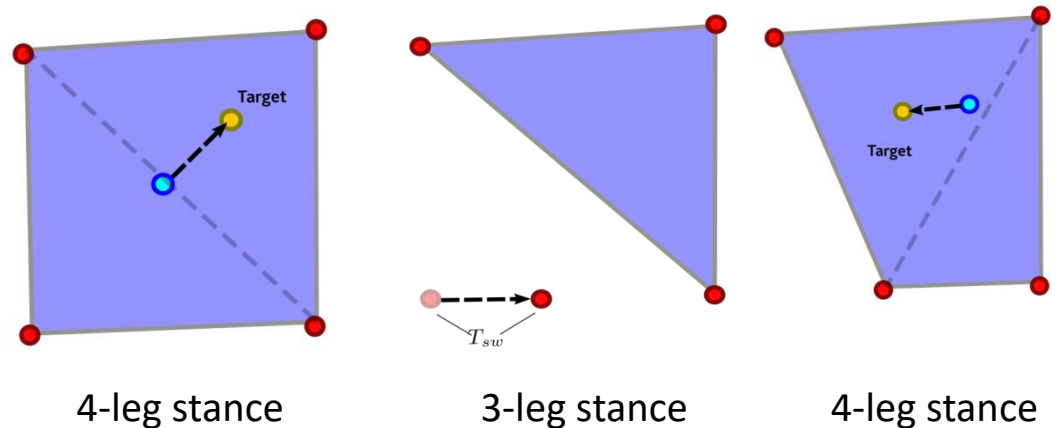
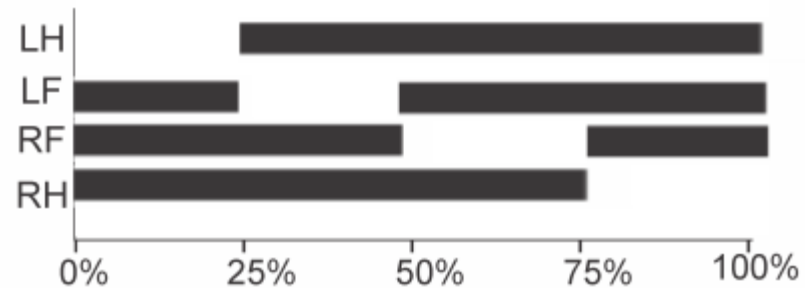
Humanoids



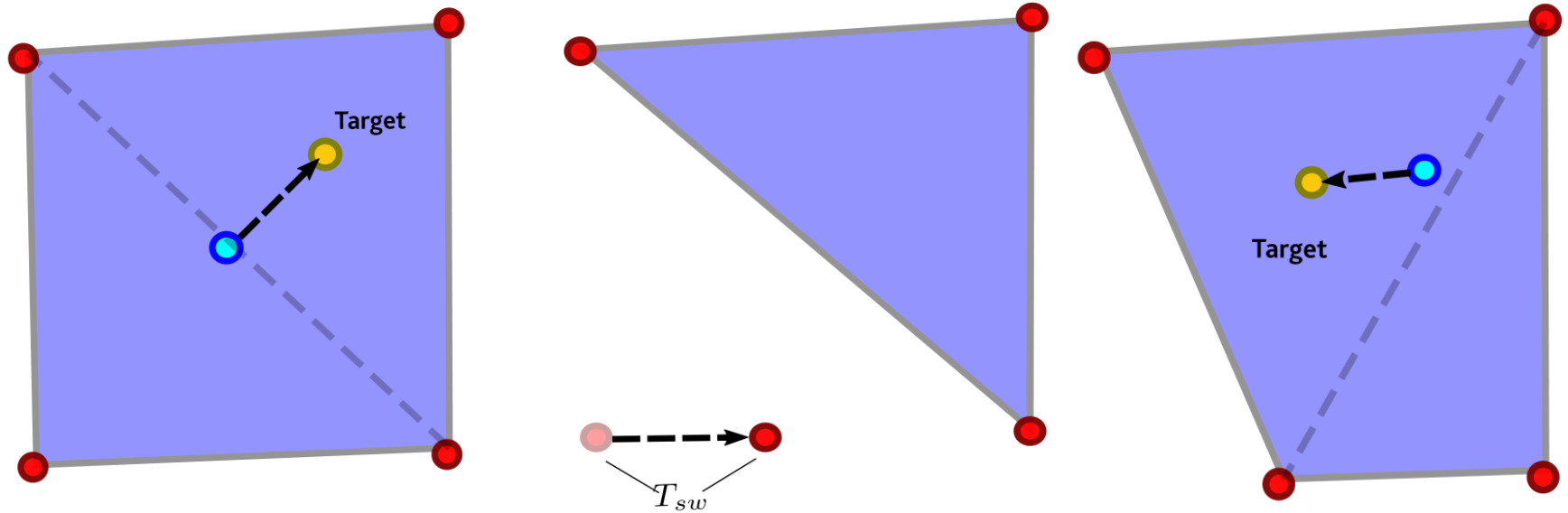
- Support Polygon continuously changes with contact condition

Quadrupeds

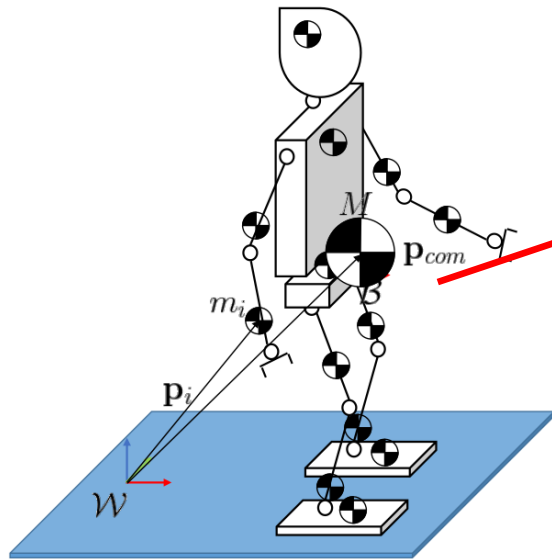
Gait scheduler: crawl example



Support Polygon: Heuristic walk example



Robot Center of Mass (CoM)



$$p_{com} = \frac{\sum_{i=0}^m p_{f,i} m_i}{\sum_{i=0}^m m_i},$$

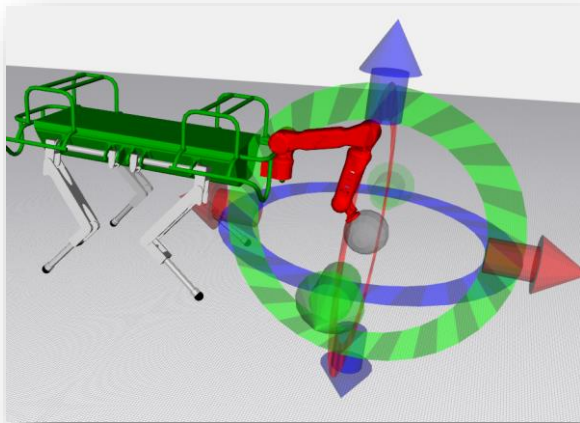
m_i → Mass of link i

$p_{f,i}$ = position of the CoM of link i

$$m_{tot} = \sum_{i=0}^m m_i.$$

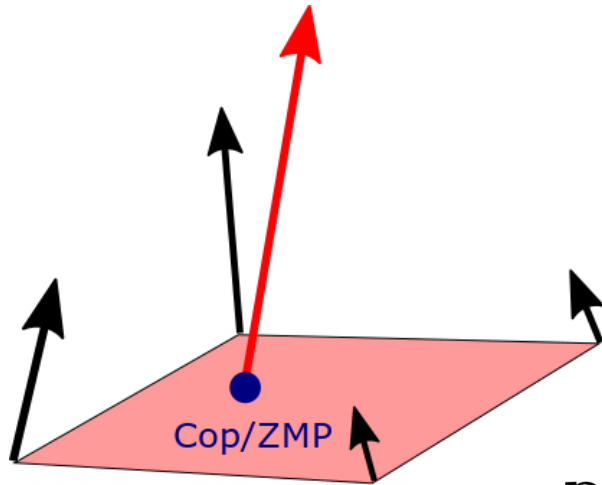
→ Total robot mass

- CoM is a **moving point** and depends on the joint configuration
- Why do we care about CoM?



- Closing the loop and planning trajectory for the CoM is crucial when you have decentralized loads (e.g. an arm mounted on the robot)

Center of Pressure



- Assumes all contact points lie on a common plane
- Point where resultant of contact forces is applied
- Also called **Zero Moment Point**
- Constrained inside the convex hull of stance feet (support polygon)

$$\mathbf{p}_{\text{CoP}} = \frac{\sum_{i=1}^{n_c} f_{z,i} \mathbf{p}_{f,i}}{\sum_{i=1}^{n_c} f_{z,i}},$$

position of foot/contact
i on the ground plane

normal force at foot i

- for point contacts on a flat ground, this is the expression usually used.

$$x_{\text{CoP}} = \frac{\sum_i f_{z,i} x_i}{\sum_i f_{z,i}}, \quad y_{\text{CoP}} = \frac{\sum_i f_{z,i} y_i}{\sum_i f_{z,i}}.$$

- If the feet are non-coplanar, there is generally no unique single CoP

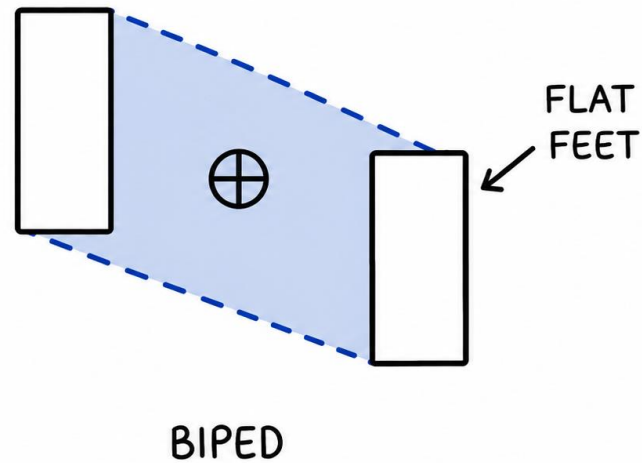
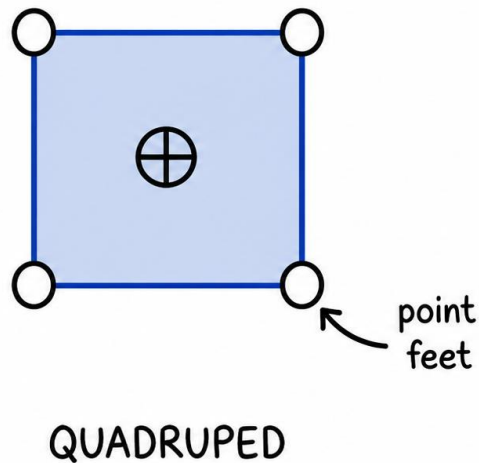
- better to talk about the **resultant contact wrench** (about some reference point O)

$$\mathbf{f}_R = \sum_i \mathbf{f}_i,$$

$$\boldsymbol{\tau}_O = \sum_i (\mathbf{p}_i - \mathbf{p}_O) \times \mathbf{f}_i,$$

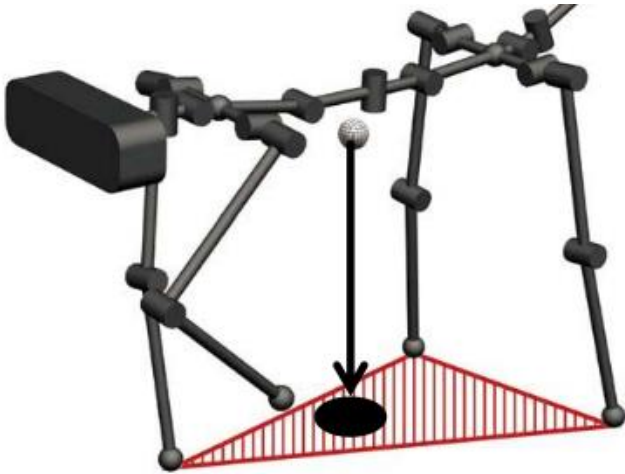
Locomotion balance criteria

- Avoid falling, tipping over $\neq$ classical control stability definition
- **Static stability criterion:** keep the CoM projection inside the support polygon.
- **Dynamic stability criterion:** keep the ZMP inside the support polygon.



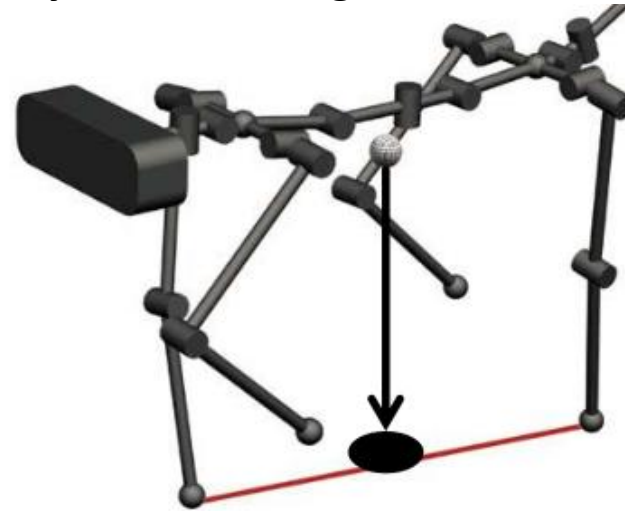
Static vs. Dynamic Stability

Statically stable



- Bodyweight supported by at least three legs
- Even if all joints 'freeze' instantaneously, the robot will not fall
- Safe, slow and inefficient

Dynamic walking



- The robot will fall if not continuously moving
- Less than three legs can be in ground contact
- Fast, efficient and demanding for actuation and control

Outline

Brief History of legged Robots

Floating Base Robots

Basic Concepts in Legged Locomotion

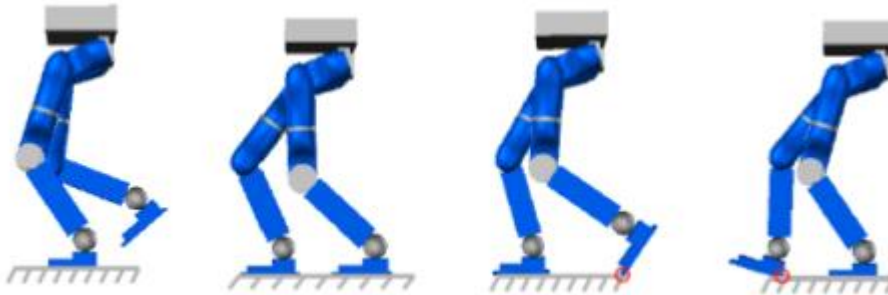
Modeling Legged robots

Towards Newton-Euler equations

Quasi-static Trunk Controller

Modeling legged robots

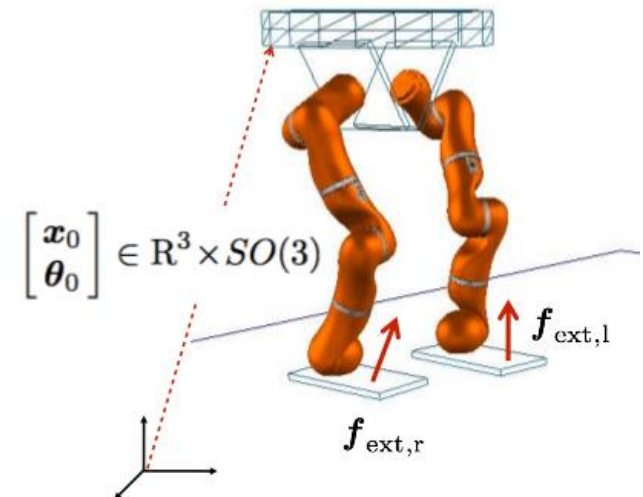
Minimal coordinates



- It looks at these contact configurations as different fixed-base robots, each with a specific kinematic and dynamic models

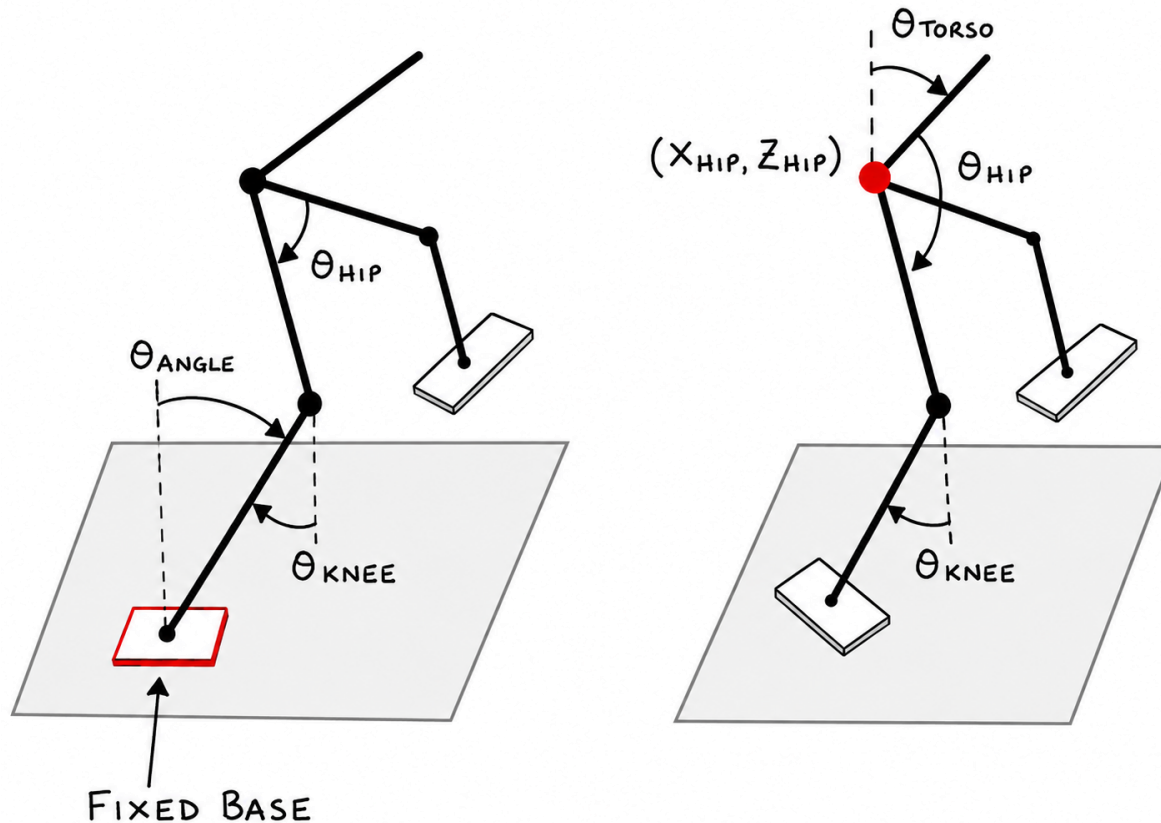
Excess of coordinates

- Considers a single floating-base system with limbs that may establish contacts



Modeling legged robots : minimal coordinates

- Get the minimum amount of coordinates for each mode



Stance Mode

$$q_{st} = [\theta_{ANKLE}, \theta_{KNEE}, \theta_{HIP}]$$

Flight Mode

$$q_{fl} = [x_{HIP}, z_{HIP}, \theta_{TORSO}, \theta_{HIP}, \theta_{KNEE}]$$

Minimal coordinates



Simple dynamics

$$M(q_J)\ddot{q}_J + R(q_J, \dot{q}_J) = \tau_J, \\ q_J \in \mathbb{R}^m.$$



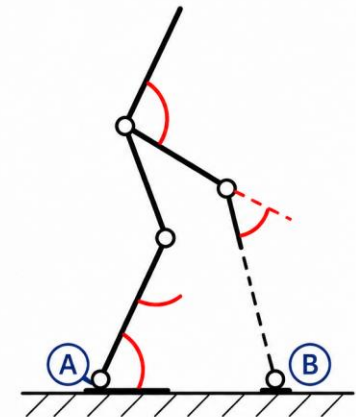
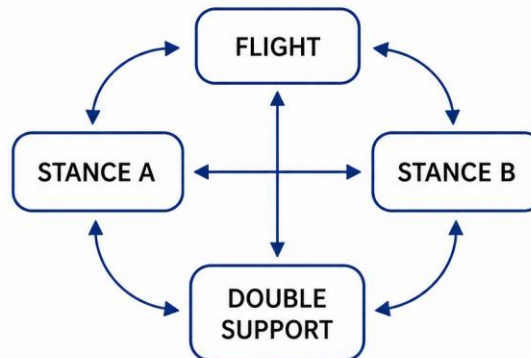
The parameterization implicitly takes into account the contact constraints.



If a foot is placed on the ground, the system loses 6 DoFs, and we would need a different set of generalized coordinates.

BIPED

- 4 modes
- 12 transitions



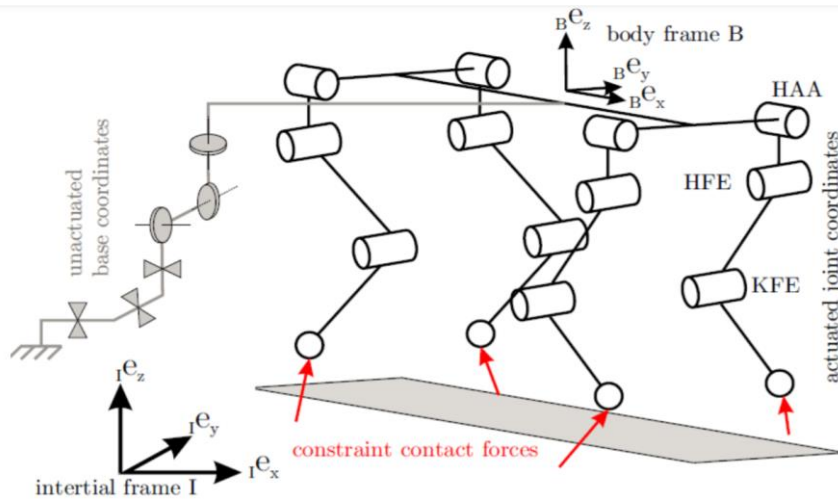
QUADRUPEDS: 16 modes ...

Complex

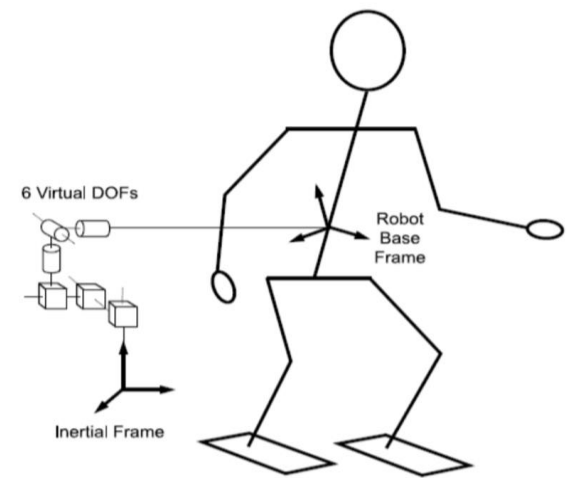
Modeling legged robots : excess of coordinates

- **Actuated** Joint angles are not enough to describe the robot configuration
- In a legged robot the degrees of freedom of the system include both the actuated joints and the six (minimum) degrees of freedom which define the position and orientation of the robot in space (q_b)
- Legged robots can be modeled as fixed-base robots adding add a 6DoF **virtual** kinematic chain between the World W and the base B

$$\mathbf{q} = \begin{bmatrix} \mathbf{q}_b \\ \mathbf{q}_j \end{bmatrix} \text{ with } \mathbf{q}_b = \begin{bmatrix} \mathbf{p}_b \\ \Phi_b \end{bmatrix} \in \mathbb{R}^3 \times SO(3) \text{ and } \mathbf{q}_j \in \mathbb{R}^n, \boldsymbol{\tau} \in \mathbb{R}^n$$



quadruped



biped

- The pose of each link can be computed from the position/orientation of the base (pose) q_b and the actuated joints configuration q_j

Floating Base Kinematics

- The unactuated base is free in translation and rotation

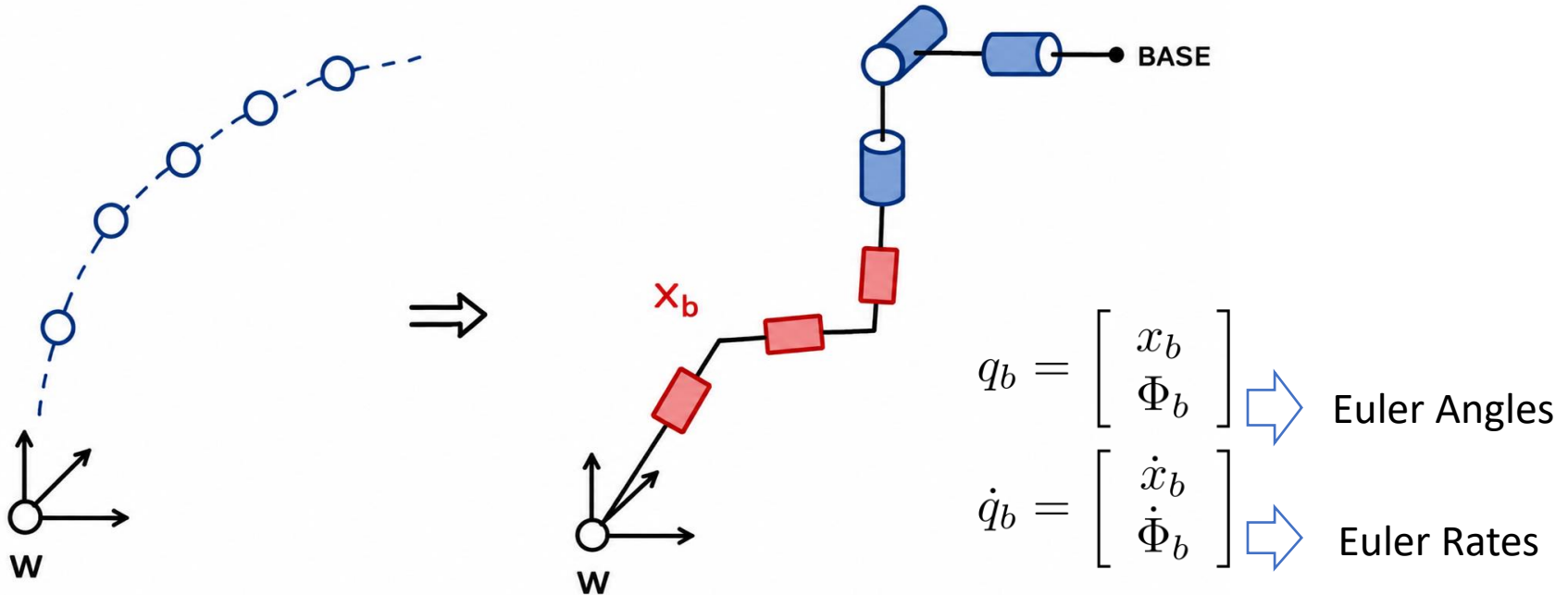
$$\mathbf{q}_b = \begin{pmatrix} \mathbf{q}_{b_P} \\ \mathbf{q}_{b_R} \end{pmatrix} \in \mathbb{R}^3 \times SO(3)$$

- Minimal number of generalized coordinates for the base is $n_b=6$
- The rotation $\mathbf{q}_{b_R}$ can be parameterized using different representations
 - Rotation matrix [9 params]
 - Euler Angles [3 params]
 - Quaternions [4]
 - Angle-axis [4]

Different representations of orientation

- Depending on the way we parametrize the orientation of B we can have different generalized velocities:

A) Virtual Joints: $\begin{cases} 3 \text{ PRISMATIC} \rightarrow \text{TRANSLATION} \in \mathbb{R}^3 \\ 3 \text{ ROTATIONAL} \rightarrow \text{ORIENTATION} \in \mathbb{R}^3 \end{cases}$



Base Jacobian with virtual joints

- The base Jacobian maps base generalized velocities (with Euler angle rates) to the spatial velocity of the base

$${}_wV_b = \begin{bmatrix} \dot{x}_b \\ \omega_b \end{bmatrix} = {}_wJ_b \dot{q}_b, \quad \dot{q}_b = \begin{bmatrix} \dot{x}_b \\ \dot{\Phi} \end{bmatrix}.$$

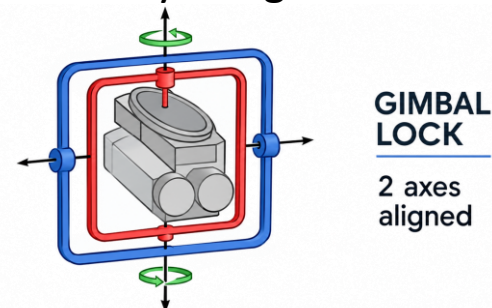
↓

$${}_wJ_b = \begin{bmatrix} I & 0 \\ 0 & J_\omega \end{bmatrix}. \quad J_\omega \text{ maps Euler rates } \dot{\Phi} \text{ into } \omega.$$

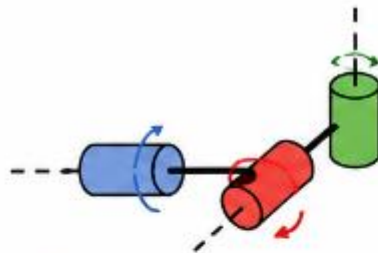
✓ $q_b = \begin{bmatrix} x_b \\ \Phi \end{bmatrix}, \quad \dot{q}_b = \frac{d}{dt} q_b.$ Direct relation between virtual-chain kinematics and generalized velocities

✓ $q_{b,k+1} = q_{b,k} + \Delta T \dot{q}_{b,k+1}.$ Easy to numerically integrate

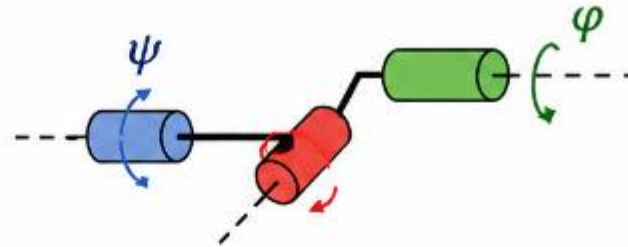
✗ Singularity associated to Euler angles



Gimbal lock: example of singularity with ZYX convention

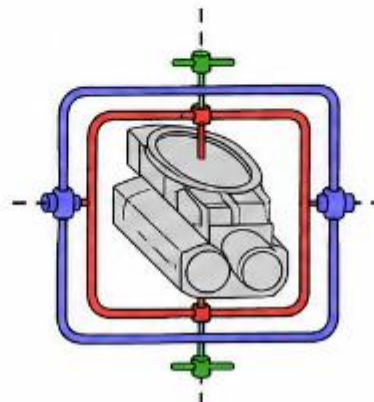
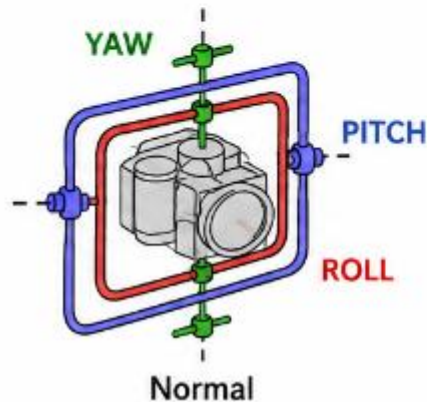


$$\begin{aligned}\psi &= 0 \\ \theta &= 0 \\ \varphi &= 0\end{aligned}$$



$$\begin{aligned}\psi &= ? \\ \theta &= -\pi/2 \\ \varphi &= ?\end{aligned}$$

Cannot distinguish between ψ and $(-\varphi)$



- more sets of Euler angles correspond to the same rotation matrix. For example

$$R\left(\frac{\pi}{4}, \frac{\pi}{2}, 0\right) = R\left(0, \frac{\pi}{2}, -\frac{\pi}{4}\right) = R\left(\frac{\pi}{8}, 0, -\frac{\pi}{8}\right).$$

Base twist as generalized base velocity

B) Free floating body:

$$\dot{q}_b = \begin{bmatrix} \dot{x}_b \\ \omega_b \end{bmatrix}, q_b = \begin{bmatrix} x_b \\ ? \end{bmatrix} \Rightarrow \text{depends on the parametrization}$$



Base Jacobian is the identity:

$$\dot{q}_b = {}^W V_B \implies {}^W J_B = I_{6 \times 6}. \quad \text{where } {}^W V_B \text{ is the spatial velocity}$$

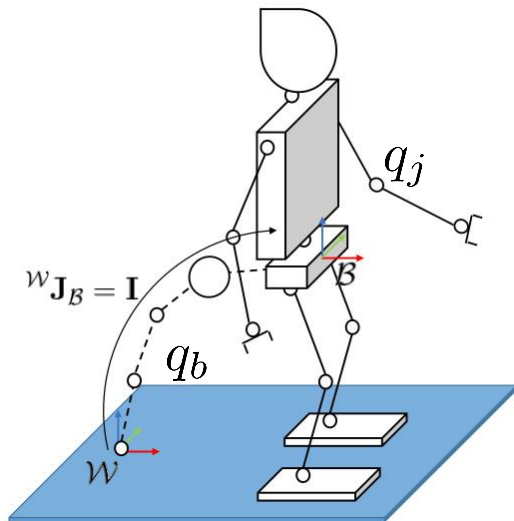
- If ? = quaternion Q_b :



Non-minimal representation, no singularities



Pose and velocity do not have the same size.



$$q_b = \begin{bmatrix} x_b \\ Q_b \end{bmatrix} \in \mathbb{R}^7,$$

$$\dot{q}_b = \begin{bmatrix} \dot{x}_b \\ \omega_b \end{bmatrix} \in \mathbb{R}^6.$$

Vector dimensions using quaternions

- Therefore, if we use quaternions to represent orientation:

ROBOT CONFIGURATION: $q = (q_b, q_J) \in \mathbb{R}^{n+7},$

$\searrow$

$q_b = (x_b, Q_b) \in \mathbb{R}^7.$

Slight liberty of notation since $\dot{q} \neq \frac{d}{dt}q.$

ROBOT VELOCITY: $\dot{q} = (\dot{q}_b, \dot{q}_J) \in \mathbb{R}^{n+6},$

$\uparrow$

$\searrow$

$\dot{q}_b = (\dot{x}_b, \omega_b) \in \mathbb{R}^6.$

ROBOT ACCELERATION: $\ddot{q} = (\ddot{q}_b, \ddot{q}_J) \in \mathbb{R}^{n+6},$

$\searrow$

$\ddot{q}_b = (\ddot{x}_b, \dot{\omega}_b) \in \mathbb{R}^6.$

Differential Kinematics: Geometric Jacobian

- Jacobian maps generalized velocities to time-derivatives of the end-effector configuration representation
- The associated partial differentiations of the end-effector configuration depends on the selected parameterization for the rotation
- Exist a unique Jacobian that relates the generalized velocity $\dot{\mathbf{q}}$ to the (spatial) velocity of the end-effector
$$\begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \mathbf{J}(\mathbf{q})$$
- $\mathbf{J}$ is the **geometric** Jacobian and it has the dimension $6 \times (n_b + n_j)$
- From basic kinematics we know that the velocity of a point C can be calculated from the velocity of a point B and the relative velocity between B and C:

$$\begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = T_B + T_{BC}$$

- Then geometric Jacobians can be simply added if are represented with respect to the same reference frame.

$$\mathbf{J}_C = \mathbf{J}_B + \mathbf{J}_{BC},$$

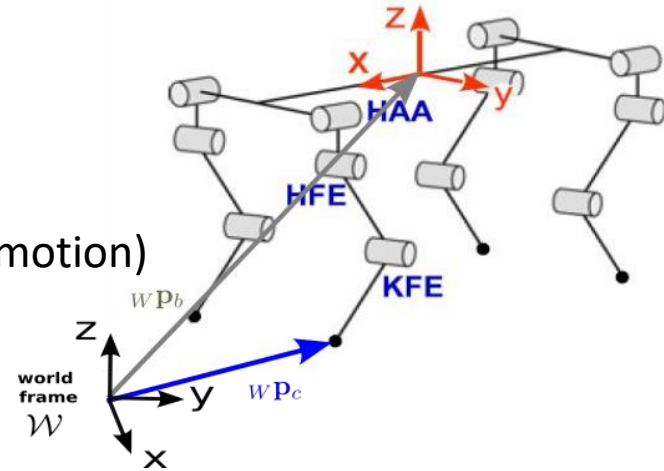
Derivation of the geometric Jacobian for floating base robots

- Position of an arbitrary point on the robot

$${}^w\mathbf{p}_c = {}^w\mathbf{p}_b + {}^w\mathbf{R}_b(\mathbf{q}) {}^b\mathbf{p}_{b,c}$$

- Velocity of this point (quadruped case m=3, only linear motion)

$$\begin{aligned} {}^w\mathbf{v}_c &= {}^w\mathbf{v}_b + {}^w\dot{\mathbf{R}}_b \cdot {}^b\mathbf{p}_{b,c} + {}^w\mathbf{R}_b \cdot {}^b\dot{\mathbf{p}}_{b,c} \\ &= {}^w\mathbf{v}_b + {}^w\boldsymbol{\omega}_b \times {}^w\mathbf{R}_b \cdot {}^b\mathbf{p}_{b,c} + {}^w\mathbf{R}_b \cdot {}^b\dot{\mathbf{p}}_{b,c} \\ &= {}^w\mathbf{v}_b - {}^w\mathbf{R}_b \cdot {}^b\mathbf{p}_{b,c} \times {}^w\boldsymbol{\omega}_b + {}^w\mathbf{R}_b \cdot {}^b\dot{\mathbf{p}}_{b,c} \\ &= {}^w\mathbf{v}_b - {}^w\mathbf{R}_b \cdot [{}^b\mathbf{p}_{b,c}]_{\times} \cdot {}^w\boldsymbol{\omega}_b + {}^w\mathbf{R}_b \cdot {}^b\mathbf{J}_{P_c}(\mathbf{q}_j) \cdot \dot{\mathbf{q}}_j \\ &= \underbrace{\begin{bmatrix} \mathbf{I}_{3 \times 3} & -{}^w\mathbf{R}_b \cdot [{}^b\mathbf{p}_{b,c}]_{\times} \end{bmatrix}}_{{}^w\mathbf{J}_c(\mathbf{q})} \begin{bmatrix} {}^w\mathbf{R}_b \cdot {}^b\mathbf{J}_{P_c}(\mathbf{q}_j) \end{bmatrix} \cdot \dot{\mathbf{q}}. \end{aligned}$$



$$\mathbf{J}_c = \begin{bmatrix} \mathbf{J}_{c,b} & \mathbf{J}_{c,j} \end{bmatrix} \in \mathbb{R}^{3 \times (n+6)}$$

$\mathbf{J}_{c,b}$ indicates the relation between base motion and foot

$\mathbf{J}_{c,j}$ indicates the relation between joint motion and foot

Derivation of the geometric Jacobian for floating base robots

- If we attach a frame at p_f (case of $m=6$, humanoids), we can derive a similar mapping for angular velocities

$$J_c = \begin{bmatrix} J_{c_b} & J_{c_j} \end{bmatrix} \in \mathbb{R}^{6 \times (n+6)}$$


$$J_{c_b} = \begin{bmatrix} I_{3 \times 3} & -[{}^w\mathbf{p}_{c,i} - {}^w\mathbf{p}_b]_x \\ 0_{3 \times 3} & I_{3 \times 3} \end{bmatrix} \in \mathbb{R}^{6 \times 6}$$

$$J_{f_j} = \frac{\partial({}^w\mathbf{p}_{f,i} - {}^w\mathbf{p}_b)}{\partial q_j} \in \mathbb{R}^{6 \times n}$$

- Hence, the mapping from generalized velocities $\dot{q}$ to the operational space of a frame attached to p_f is realized by the Jacobian:

$$\begin{bmatrix} {}^w\mathbf{v}_c \\ \boldsymbol{\omega}_c \end{bmatrix} = J_c \dot{\mathbf{q}} \quad \text{with} \quad \dot{\mathbf{q}} = \begin{bmatrix} \dot{x}_b \\ \boldsymbol{\omega}_b \\ \mathbf{q}_j \end{bmatrix}$$

- Henceforth, when the frame is not specified the world (inertial) frame is assumed

Dynamics of fixed-base robots in contact

- Multi-body dynamics is formulated as:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} + \mathbf{J}_c(\mathbf{q})^T \mathbf{f}_c$$

- Different methods exist to compute the so-called Equations of Motion (EoM):
Newtonian and/or Lagrangian mechanics formulations equivalent descriptions of the dynamics
 - **Recursive Newton-Euler method:** applies the principles of conservation of linear and angular momentum for all links of a robot
 - **Lagrangian method:** scalar energy-based functions over the the space of generalized coordinates

External Forces

- To account for multiple external forces in EoM expressed in generalized coordinates, we can project Cartesian forces and moments onto the space of generalized coordinates using appropriate Jacobian matrices

$$\boldsymbol{\tau}_{F,ext} = \sum_{j=1}^{n_c} \mathbf{J}_{P,j}^T \mathbf{f}_j$$

$$\boldsymbol{\tau}_{T,ext} = \sum_{k=1}^{n_c} \mathbf{J}_{R,k}^T \mathbf{m}_k$$

- If I have multiple contacts I can stack them in one Jacobian multiplying the vector of all the contact forces

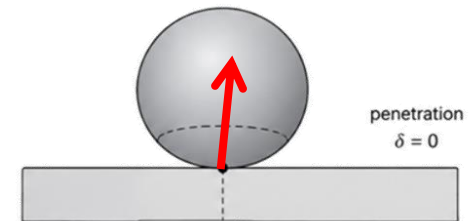
$$\mathbf{J}_c = \begin{bmatrix} \mathbf{J}_{C_1} \\ \vdots \\ \mathbf{J}_{C_{n_c}} \end{bmatrix} \in \mathbb{R}^{3n_c \times n_n}$$

Contact Modeling

- There exist two fundamentally different methods to model contacts:
- The **hard contact method** treats the contact as a kinematic constraint.
 - No interpenetration allowed
 - Contact force acts only when bodies touch
 - Instantaneous impact (impulse)
- The **soft contact method** models the interaction by force elements (i.e. spring-damper) where the force is only a function of the location and velocity of the point in contact.
 - A “penetration” with the environment is needed to have a force.
 - Contact force increases with penetration

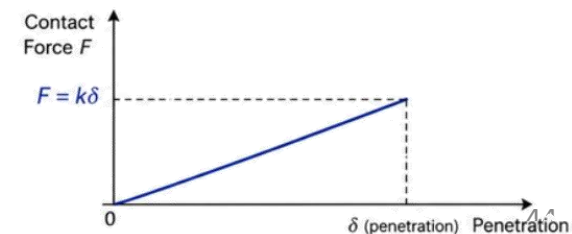
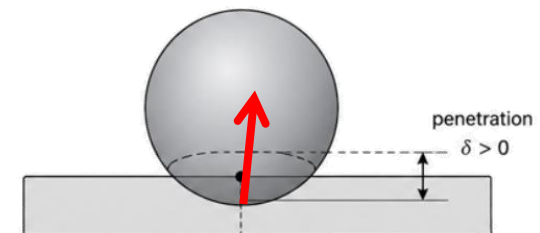
HARD CONTACT MODEL

No penetration



SOFT CONTACT MODEL

Allowable penetration

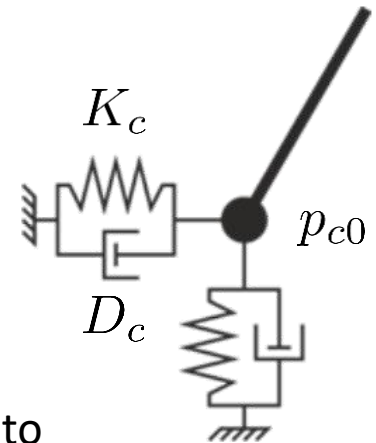


Soft Contact Model

- We typically identify the point when first making contact with the environment as $\mathbf{p}_{c0}$.
- When using a linear spring-damper model for the environment, the contact force (exerted by the robot on its environment) is

$$\mathbf{f} = K_c(\mathbf{p}_{c0} - \mathbf{p}_c) - D_c\dot{\mathbf{p}}_c$$

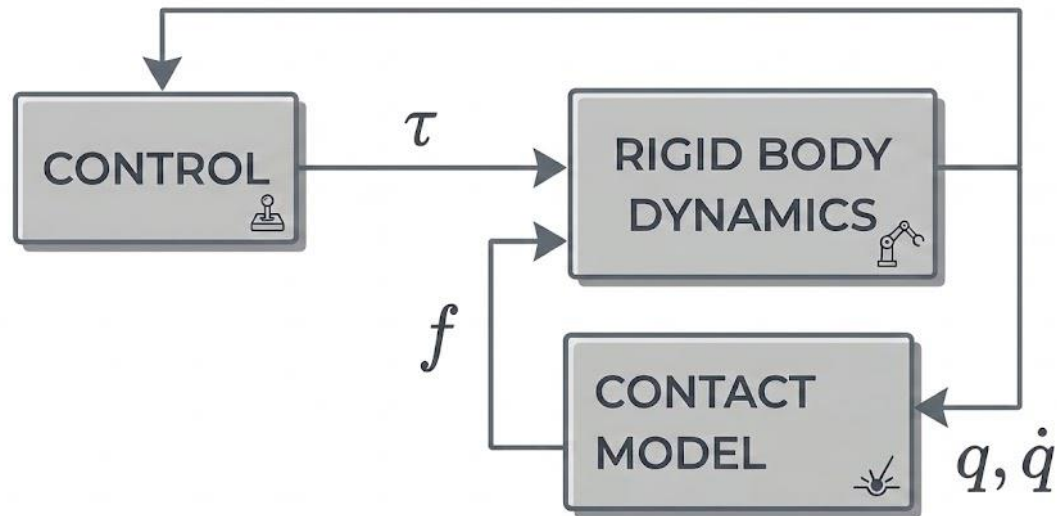
$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} + \mathbf{J}_c(\mathbf{q})^T \mathbf{f}$$



- Limits: finding physically correct spring and damping parameters to simulate the system dynamics is almost impossible
- Combined differential equations for multi-body system and contacts become very stiff (slow multi-body dynamics and very fast contact dynamics).
- Solving such problems either result in poor speed or low accuracy

Soft Contact Model

- Generic law $\mathbf{f} = f(\mathbf{p}_c, \dot{\mathbf{p}}_c)$



Hard Contact Model

- Contacts between the robot and its environment can be modeled as **kinematic constraints**
- If a point (with attached coordinate frame C_i and position $\mathbf{p}_c$) is in contact with the environment, it is not allowed to move anymore and imposes three constraints:

$$\mathbf{p}_c = \text{const} \quad \Rightarrow \quad \dot{\mathbf{p}}_c = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \ddot{\mathbf{p}}_c = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

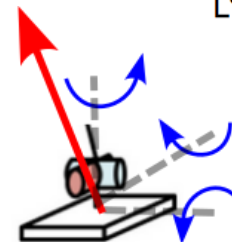
- These Constraints are expressed as a function of the generalized velocities and accelerations using the contact point Jacobian

$$\dot{\mathbf{p}}_c = \mathbf{J}_c \dot{\mathbf{q}} = 0$$

$$\ddot{\mathbf{p}}_c = \mathbf{J}_c \ddot{\mathbf{q}} + \dot{\mathbf{J}}_c \dot{\mathbf{q}} = 0$$

- For all the active contacts, the constraints are simply stacked to: $\mathbf{J}_c = \begin{bmatrix} \mathbf{J}_{C_1} \\ \vdots \\ \mathbf{J}_{C_{n_c}} \end{bmatrix} \in \mathbb{R}^{3n_c \times n_n}$

- The same formulation can be made for rotational constraints in case of and extended contact area



Contact Consistent Inverse Kinematics

- Applying inverse kinematics to systems in contact answers the question how to move individual joints in order to achieve certain task-space motion **without** violating contact constraints.
- If we have **redundancy** the system can be moved without violating the contact constraints!

$$\begin{aligned} 0 = \dot{\mathbf{p}}_c = \mathbf{J}_c \dot{\mathbf{q}} &\implies \dot{\mathbf{q}} = \mathbf{J}_c^+ 0 + \mathbf{N}_c \dot{\mathbf{q}}_0 = \mathbf{N}_c \dot{\mathbf{q}}_0 \\ 0 = \ddot{\mathbf{p}}_c = \mathbf{J}_c \ddot{\mathbf{q}} + \dot{\mathbf{J}}_c \dot{\mathbf{q}} &\implies \ddot{\mathbf{q}} = \mathbf{J}_c^+ \left(-\dot{\mathbf{J}}_c \dot{\mathbf{q}} \right) + \mathbf{N}_c \ddot{\mathbf{q}}_0 \end{aligned}$$

motion of the system in contact

- However, for floating base robots the base is **underactuated**
- The base part of the Contact Jacobian tells us, how a system can move.

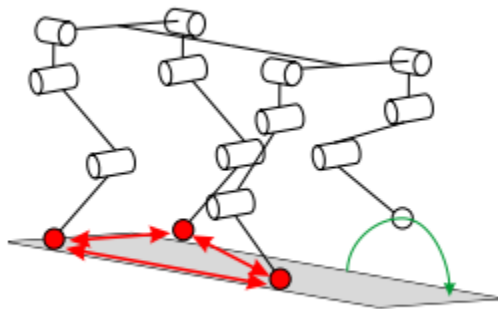
$$\mathbf{J}_c = \begin{bmatrix} \boxed{\mathbf{J}_{c,b}} & \mathbf{J}_{c,j} \end{bmatrix} \in \mathbb{R}^{3 \times (n+6)}$$

Relation between base motion and constraints

- If $\text{rank}(\mathbf{J}_{c,b}) = 6$: the system features enough constraints such that the base motion can be controlled from joint motion: the base becomes **fully** controllable

Point contacts quadruped

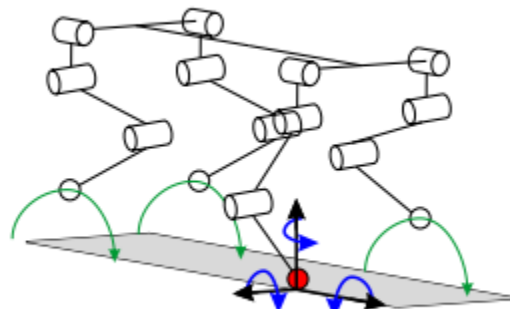
- The difference between the number of independent contact constraints $\text{rank}(\mathbf{J}_c)$ and the number of base constraints $\text{rank}(\mathbf{J}_{c,b})$ is the number of internal kinematic constraints that must be fulfilled (e.g. keep relative feet distance).
- Depending on the number and arrangement of point contacts, a quadruped is fully-constrained (a) or under-constrained (b)



(a) Fully-constraints

$\text{rank}(\mathbf{J}_c) = 9, \text{rank}(\mathbf{J}_{c,b})=6$

Full base actuation + 3 internal constraints (3 feet cannot be moved one related to the other)



(b) Under-constrained

$\text{rank}(\mathbf{J}_c) = 6, \text{rank}(\mathbf{J}_{c,b})=5$

Base is **underactuated**: cannot control orientation about the line connecting supporting feet + 1 internal constraint

Floating base dynamics in contact

- EoM from fixed base robots

$$\mathbf{M}(\mathbf{q}_j)\ddot{\mathbf{q}}_j + \underbrace{\mathbf{C}(\mathbf{q}_j, \dot{\mathbf{q}}_j)\dot{\mathbf{q}}_j}_{\mathbf{c}(\mathbf{q}_j, \dot{\mathbf{q}}_j)} + \mathbf{g}(\mathbf{q}_j) = \boldsymbol{\tau}$$

- Not all joints are actuated, base is underactuated

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \mathbf{S}\boldsymbol{\tau}$$

The selection matrix selects the actuated joints

$$\mathbf{S} = [\mathbf{0}_{n \times 6} \quad \mathbf{I}_{n \times n}]$$

$$\mathbf{q}_j = \mathbf{S}\mathbf{q}$$

- In order to control also the unactuated base coordinates $\mathbf{q}_b$, external forces $\mathbf{f}_c$ are necessary.

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \underbrace{\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q})}_{\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})} = \mathbf{S}\boldsymbol{\tau} + \mathbf{J}_c^T \mathbf{f}$$



This implicitly involves contact constraints

$$\mathbf{x}_c = \bar{\mathbf{x}} \implies \mathbf{J}_c \dot{\mathbf{q}} = 0 \implies \mathbf{J}_c \ddot{\mathbf{q}} + \dot{\mathbf{J}}_c \dot{\mathbf{q}} = 0$$

- Generalized coordinates DON'T correspond to the degrees of freedom (there are contact constraints that limit them)
- Minimal coordinates = correspond to degrees of freedom (but require to switch the set of coordinates depending on contact state)

Summary minimal/excess of coordinates

Minimal coordinates

- different # DOFs in each contact mode

$$q_{fl} = [x_{hip} \quad z_{hip} \quad \theta_{hip} \quad \theta_{knee} \quad \theta_{torso}] ,$$

$$q_{st} = [\theta_{ankle} \quad \theta_{knee} \quad \theta_{hip}] .$$

- Dealing with transitions between modes
- Contact constraints are embedded in the choice of coordinates
- suitable for systems with simple contact scenarios

Excess of coordinates

- same #DOFs (use only one coordinate convention)

⇒ simplified code

- different # of constraints

$$\begin{bmatrix} M & -J_c^T \\ J_c & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ f_c \end{bmatrix} = \begin{bmatrix} S^T \tau - h \\ -J_c \dot{q} \end{bmatrix} .$$

(J_c changes)

Contact Force estimation from constraints

- In matrix form:

$$\begin{matrix} (1) \\ (2) \end{matrix} \begin{bmatrix} M & -J_c^T \\ J_c & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ f_c \end{bmatrix} = \begin{bmatrix} S^T \tau - h \\ -\dot{J}_c \dot{q} \end{bmatrix}.$$

- External forces from constraints. Solve (1) for $\ddot{q}$ and insert it in contact constraint equation (2)

$$M\ddot{q} + h = S^T \tau + J_c^T f,$$

$$\ddot{q} = M^{-1} (-h + S^T \tau + J_c^T f), \quad \longrightarrow \quad J_c \ddot{q} + \dot{J}_c \dot{q} = 0$$

$$J_c M^{-1} (-h + S^T \tau + J_c^T f) + \dot{J}_c \dot{q} = 0.$$

- Solve for contact forces:

$$J_c M^{-1} J_c^T f + J_c M^{-1} (-h + S^T \tau) + \dot{J}_c \dot{q} = 0,$$

$$(3) \quad f = \left(J_c M^{-1} J_c^T \right)^{-1} \left[-\dot{J}_c \dot{q} + J_c M^{-1} (h - S^T \tau) \right].$$

 Λ_c Reflected inertia at the end-effector

Derivation of Contact consistent joint space dynamics

- (3) provides direct access to estimated ground reaction forces solely based on the description of the multi-body system dynamics and without any further contact force sensor!
- Plugging $\mathbf{f}$ (3) back into dynamics (1):

$$M\ddot{\mathbf{q}} + \mathbf{h} = \mathbf{S}^T \boldsymbol{\tau} + \mathbf{J}^T (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1} \left[-\dot{\mathbf{J}} \dot{\mathbf{q}} + \mathbf{J} \mathbf{M}^{-1} (\mathbf{h} - \mathbf{S}^T \boldsymbol{\tau}) \right].$$

- Expanding:

$$(4) \quad M\ddot{\mathbf{q}} + \mathbf{h} = \mathbf{S}^T \boldsymbol{\tau} - \mathbf{J}^T (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1} \dot{\mathbf{J}} \dot{\mathbf{q}} + \mathbf{J}^T (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1} \mathbf{J} \mathbf{M}^{-1} \mathbf{h} - \mathbf{J}^T (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1} \mathbf{J} \mathbf{M}^{-1} \mathbf{S}^T \boldsymbol{\tau}.$$

- Recall: For a matrix $\mathbf{A}$, the pseudoinverse and weighted pseudoinverse are

$$\mathbf{A}^* = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T,$$

$$\mathbf{A}_W^* = (\mathbf{A}^T \mathbf{W} \mathbf{A})^{-1} \mathbf{A}^T \mathbf{W}.$$

For $\mathbf{A} = \mathbf{J}^T$ and $\mathbf{W} = \mathbf{M}^{-1}$:

$$\mathbf{J}_M^{T*} = (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1} \mathbf{J} \mathbf{M}^{-1},$$

weighted (dynamically consistent)
pseudoinverse of $\mathbf{J}^T$

Derivation of Contact consistent joint space dynamics

$$M\ddot{q} + \underbrace{h}_{\text{blue}} = \underbrace{S^T \tau}_{\text{green}} + J^T (JM^{-1}J^T)^{-1} \dot{J}\dot{q} + J^T (JM^{-1}J^T)^{-1} JM^{-1} \underbrace{h}_{\text{blue}} - J^T (JM^{-1}J^T)^{-1} JM^{-1} \underbrace{S^T \tau}_{\text{green}}.$$

- Now collect h and $S^T \tau$

$$M\ddot{q} + h \underbrace{\left[I - J^T J_M^{T\#} \right]}_{N_M} + J^T \underbrace{\left(JM^{-1}J^T \right)^{-1}}_{\Lambda} (\dot{J}\dot{q}) = \underbrace{\left[I - J^T J_M^{T\#} \right]}_{N_M} S^T \tau$$

- Defining the null-space projector N_M

$$N_M = I - J_M^T J_M^{T\#},$$

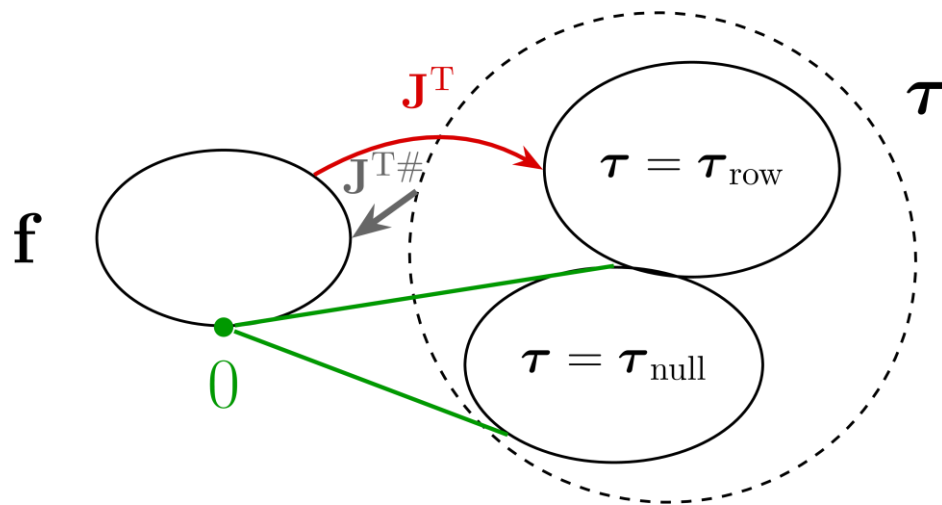
- we obtain the contact-consistent joint-space dynamics:

$$M\ddot{q} + N_M h + J^T \Lambda (\dot{J}\dot{q}) = N_M S^T \tau$$

- Contact forces have **disappeared** from the EoM!
- Note: N_M is not symmetric nor positive definite

Null space projector

- Null-space projector for J^T generates torques that do not produce any contact force (but only joint accelerations)
- Null-space is orthogonal to row space (set of torques that produce contact forces)



$$\tau_{\text{row}} = J^T J^{T\#} \tau,$$

$$\tau_{\text{null}} = \underbrace{(I - J^T J^{T\#})}_{N} \tau.$$

$$N = I - J^T J^{T\#},$$

$$\underbrace{N J^T}_{\tau_{\text{row}}} \mathbf{f} = 0$$

- Note: N_M is a linear operator that maps the dynamics onto a constrained manifold such that EOM is **independent** of contact forces. Any operator s.t. $P J^T = 0$ would also work

Constrained Consistent Forward Dynamics

- Useful to Build a **simulator**
- Given the vector of joint torques, compute the generalized accelerations.

$$\ddot{q} = M^{-1} \left(N_M (S^T - h) - J^T \Lambda J \dot{q} \right) \quad (\text{M is positive definite so we can always invert})$$

- And integrate it to get the states (e.g. by Forward Euler integration)

$$\dot{q}_{k+1} = \dot{q}_k + dT \ddot{q}_k$$

$$q_{k+1} = q_k + dT \dot{q}_{k+1} + \frac{dT^2}{2} \ddot{q}_{k+1}$$



Since we already know that the robot is a second order system. We can directly integrate position from acceleration



FE is easy to implement/understood



Computationally cheap



Low accuracy (1st order integration method)



Higher order integration methods could be used but usually FE is ok

- What about the energy lost in ground impacts?

Contact Dynamics

- **Impulse transfer at contact:** a hard contact model requires subdividing the analysis of the system dynamics into two intervals, **before** and **after** a change in the contact situation (impact)
- The characteristic of an impact is a very short duration with high peak forces that results in a rapid dissipation of energy and large accelerations
- Idealizing the process, considering the impact as an **infinitesimally** short process requires to include **instantaneous** changes in joint velocities if bodies are making contact.

$$\dot{q}^- \neq \dot{q}^+,$$

LIFT OFF: $f \downarrow 0$, $\dot{p}_c > 0$

TOUCH DOWN: $p_c = \text{const}$, $\dot{p}_c, \ddot{p}_c = 0$, $f \uparrow 0$



Collision at landing:

- foot point is immediately brought to rest
- instantaneous change in velocity to avoid penetration
- To compute it we need to integrate the dynamics of collision (inelastic impact)

Contact Dynamics

- Integration of Lagrangian dynamics on singleton

$$(1) \quad \int_{t^-}^{t^+} [M\ddot{q} + \cancel{h} + J^T f] dt = M(\dot{q}^+ - \dot{q}^-) - J^T \cancel{\mathcal{F}_c} = 0$$

All non-impulsive forces drop out.

$$\int_{t^-}^{t^+} h dt \approx h(t^+ - t^-) \rightarrow 0$$

Contact impulse

$$\mathcal{F}_c = \int_{t^-}^{t^+} f_c(t) dt.$$

- After collision the foot is at rest:

$$(2) \quad \dot{x}_c^+ = 0 = J\dot{q}^+ \quad (\text{inelastic impact})$$

- Solving for a $\dot{q}^+$ and inserting into (2):

$$(3) \quad \begin{aligned} \dot{q}^+ &= \dot{q}^- + M^{-1} J^T \mathcal{F}_c, \\ \dot{x}_f^+ &= J(\dot{q}^- + M^{-1} J^T \mathcal{F}_c) = 0. \end{aligned}$$

- Rearranging we can compute the impulse as function of $\dot{q}$:

$$\mathcal{F}_c = - (J M^{-1} J^T)^{-1} J \dot{q}^-, \quad \mathcal{F}_c^z \geq 0.$$

Contact Dynamics

- finally we compute the post-impact joint velocity replacing F_c into (3)

$$\begin{aligned}\dot{q}^+ &= \dot{q}^- + M^{-1} J^T \left[- (J M^{-1} J^T)^{-1} J \dot{q}^- \right] \\ &= \left[I - \underbrace{M^{-1} J^T (J M^{-1} J^T)^{-1} J}_{J_M^\#} \right] \dot{q}^-.\end{aligned}$$

- Defining the velocity-level null-space projector

$$N_{\text{vel}} = I - J_M^\# J = I - M^{-1} J^T (J M^{-1} J^T)^{-1} J,$$

- We obtain

$$\boxed{\dot{q}^+ = N_{\text{vel}} \dot{q}^-}.$$

- Note: This null-space projector acts at the velocity level and is different from the previous one that acts at the torque level.

Contact consistent Inverse Dynamics

- Useful to build a **controller**
- **Inverse dynamics:** given the **desired** (generalized) accelerations $\ddot{q}^d$, compute the actuation **torques** satisfying contact constraints as a function of the current state $(q, \dot{q})$ of the robot
- For legged robots the problem is ill posed because contact forces are also function of τ
- Replacing the Contact constraints in the EoM **[M. Hutter, PhD thesis, Ch2.3]:**

$$M\ddot{q} + N_M h + J^T (JM^{-1}J^T)^{-1} \dot{J}\dot{q} = N_M S^T \tau \quad - \dot{J}\dot{q} = J_c \ddot{q}$$

$$M\ddot{q} + N_M h - J^T (JM^{-1}J^T)^{-1} J\ddot{q} = N_M S^T \tau$$

- Collect $M\ddot{q}$

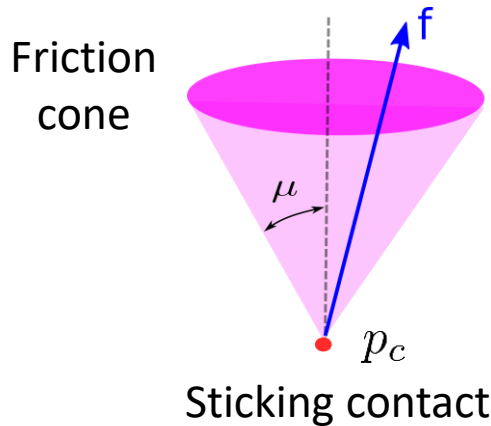
$$\underbrace{\left[I - J^T (JM^{-1}J^T)^{-1} JM^{-1} \right]}_{N_M} M\ddot{q} + N_M h = N_M S^T \tau$$

- Compute torques

$$\tau = (N_M S^T)^+ N_M (M\ddot{q} + h)$$

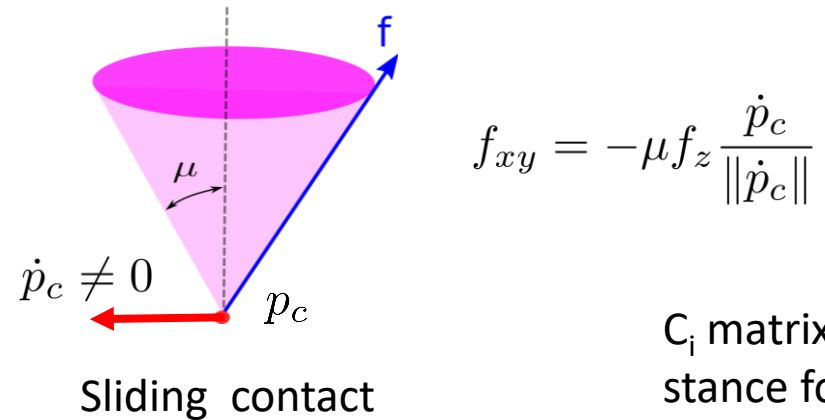
Contact Constraints: friction cone

- **Coulomb friction model:** there is a limit on the tangential force the floor can apply, beyond which the end-effector starts to slip
- This tangential force depends on the friction coefficient μ (that on its behalf it depends on the nature of both contacting surfaces)



$$\|f_{xy}\| \leq \mu f_z$$

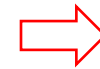
$$\dot{p}_c = 0$$



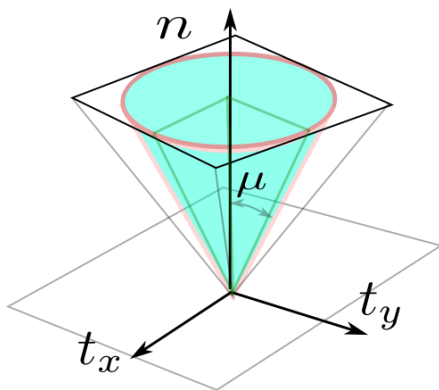
$$f_{xy} = -\mu f_z \frac{\dot{p}_c}{\|\dot{p}_c\|}$$

C_i matrix for stance foot i

Pyramid approximation (inscribed formulation)



Linear Constraint



$$\begin{cases} f_x \leq \mu f_z \\ f_y \leq \mu f_z \\ -f_x \leq \mu f_z \\ -f_y \leq \mu f_z \end{cases}$$

$$\underbrace{\begin{bmatrix} t_{x_i}^T - \mu_i n_i^T \\ t_{y_i}^T - \mu_i n_i^T \\ -t_{x_i}^T - \mu_i n_i^T \\ -t_{y_i}^T - \mu_i n_i^T \end{bmatrix}}_{C_i} f \leq \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_{\bar{d}_i}$$

Contact Constraints: unilaterality

- Rigid contact assumption: no penetration in the ground
- Cannot pull on the ground

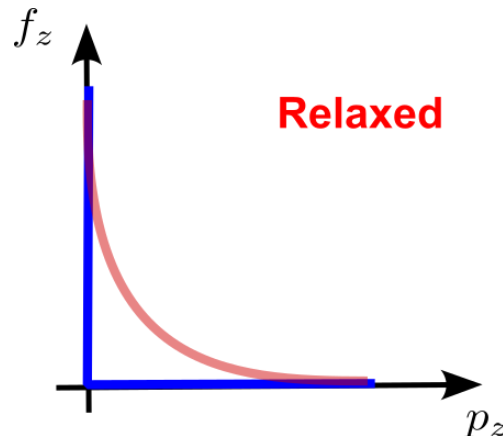
$$f_z > 0 \iff p_{c,z} = 0$$

$$f_z = 0 \iff p_{c,z} > 0$$

- **This** can be summarized with the following **complementarity condition**

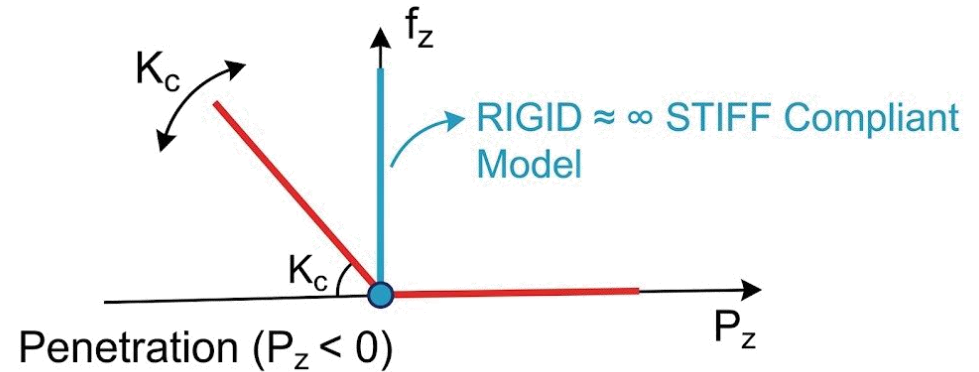
$$f_z p_{c,z} = 0 \rightarrow \text{either } p_{c,z} > 0 \text{ or } f_z > 0$$

- Complementarity constraints are pathological to deal numerically without relaxation



- Note: unilaterality constraints are naturally incorporated in the friction cone

Compliant VS Rigid contact



COMPLIANT



Easy to compute



stiff springs $\rightarrow$ ODE numerically stiff $\rightarrow$ small integration steps



SLOW SIMULATIONS



Needs tuning of contact parameters

RIGID



faster simulation and optimization



Avoids choosing poorly known contact stiffness and damping parameters



Hybrid dynamics, non smooth, Pathological to differentiate (e.g. complementarity constraint)



Use predefined sequence (time-varying)

Some more insights into the legged robots dynamics

- As in any floating base system we can partition the **underactuated** dynamics from the **actuated** one:

$$\underbrace{\begin{bmatrix} M_b(q) & M_{bj}(q) \\ M_{bj}^T(q) & M_j(q) \end{bmatrix}}_M \underbrace{\begin{bmatrix} \ddot{q}_b \\ \ddot{q}_j \end{bmatrix}}_h + \underbrace{\begin{bmatrix} c_b(q, \dot{q}) + g_b(q) \\ c_j(q, \dot{q}) + g_j(q) \end{bmatrix}}_h = \underbrace{\begin{bmatrix} 0 \\ \tau \end{bmatrix}}_J + \underbrace{\begin{bmatrix} J_{c,b}(q)^T \\ J_{c,j}(q)^T \end{bmatrix}}_J f$$

- Fact 1:** joint torques τ only affect joint coordinates! They can't have a direct influence on the part on the floating base dynamics, until the system **comes in contact with its environment**.
- Fact 2:** the floating base can be controlled only through the action of contact forces f
- Fact 3:** the contact forces f are limited by physical laws
- Fact 4:** also joint torque can be limited



Create limitations in robot ability to control its motion

- **Fact 5:** the underactuated part (first 6 rows) coincides with the **Newton - Euler equations** of motions. They link the variation of the linear/angular momentum of the whole system to the contact forces. **Assuming sufficient control authority** (sufficiently strong and actuators DOFs away from singularity), **the six equations are also sufficient conditions for planning dynamically feasible motions!**
- **Fact 6:** when supplying a certain amount of joint torque τ , the environment reacts by producing the contact forces f . Those very same forces act on the under-actuated dynamics to enable the robot to move inside the environment.



The locomotion problem can then be split into two consecutive stages:

- ① find the pattern of forces which drive the under-actuated dynamics (and resulting trajectory for CoM). This dynamics model is much simpler than the full-body model, with fewer constraints and fewer variables (**$n+6 \rightarrow 6$**)!
- ② compute the required joint torque trajectory from the **actuated** dynamics (i.e. by inverse dynamics), under the hypothesis of non sliding contacts.

References

- M. Hutter, Design and control of legged robots with compliant actuation, PhD Thesis, 2013.
- F. Aghili, A Unified Approach for Inverse and Direct Dynamics of Constrained Multibody Systems Based on Linear Projection Operator: Applications to Control and Simulation, 2005.
- A. Del Prete, PhD Thesis, 2013.

Outline

Brief History of legged Robots

Floating Base Robots

Basic Concepts in Legged Locomotion

Modeling Legged robots

Towards Newton-Euler equations

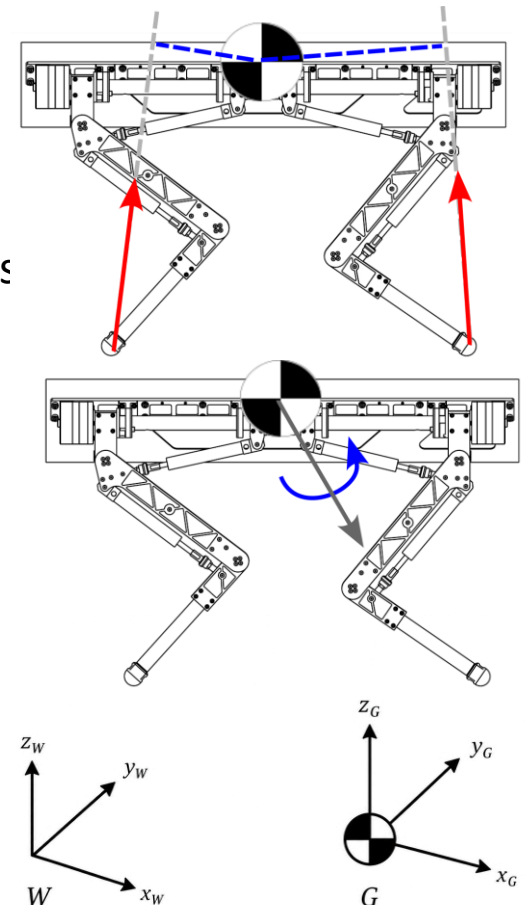
Quasi-static Trunk Controller

Total momentum (about inertial frame)

- In Euclidean algebra you need reference points (and frames) to express vectors.

${}_W \mathbf{h} \in \mathbb{R}^6$ **total momentum** = sum of all the linear and angular momenta of each rigid body expressed w.r.t. the origin of frame W and in the axes of W

- Frame W conveys 2 informations in this case:
 - **Origin**: about this point we compute the angular momentum that needs a reference point;
 - **Axes**: we decide **also** to express the vectors in these axes
- Linear forces: affect **both** linear/angular momentum.
- Moments: affect **only** angular momentum.
- The angular momentum of the system can be also expressed about other points.
- If we select a frame G attached to CoM (aligned with W), the total momentum is called **CENTROIDAL MOMENTUM**



Centroidal momentum for a robot

$$\mathbf{h}_G = \begin{bmatrix} \ell \\ \mathbf{k}_G \end{bmatrix} = \begin{bmatrix} m\dot{\mathbf{x}}_c \\ \mathbf{I}_{com}\bar{\boldsymbol{\omega}} \end{bmatrix},$$

$\dot{\mathbf{x}}_c$ is the CoM velocity expressed in G frame (attached to CoM)
 ${}_G\mathbf{k}$ is the angular momentum about G and expressed in G.
 $\mathbf{I}_{com}$ is the Average centroidal inertia
 $\bar{\boldsymbol{\omega}}$ is the locked/centroidal inertia

$\mathbf{k}_G = \sum_k^{\text{links}} (\mathbf{x}_{c,k} - \mathbf{x}_c) \times m_k \dot{\mathbf{x}}_k + \mathbf{I}_{C,k} \boldsymbol{\omega}_k,$

Konig Theorem
 $\mathbf{x}_{c,k}$ is the Position of Link's CoM
 $\mathbf{x}_c$ is the Position of Robot CoM
 $\mathbf{I}_{C,k}$ is the inertia of link k about its own CoM

State transformation for centroidal dynamics

- CoM is where gravity is applied and is paramount importance for robot balance
- The first 6 rows correspond to the underactuated dynamics of the robot but joints and base variables are still **coupled** in the Mass matrix

$$\begin{bmatrix} M_b(q) & M_{bj}(q) \\ M_{bj}^T(q) & M_j(q) \end{bmatrix}$$

- A smart change of variables can exactly “decouple” joint and floating base variables

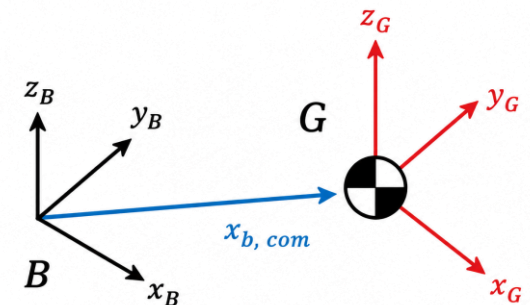
$$\dot{x}_b \rightarrow \dot{x}_c \quad \Rightarrow \text{CoM velocity}$$

$$\omega_b \rightarrow \bar{\omega} \quad \Rightarrow \text{Average angular velocity}$$

- CoM velocity depends on: 1) base linear and angular velocity and 2) on joint motion (articulated robot!)

$$\dot{\mathbf{x}}_c = \dot{\mathbf{x}}_b + \underbrace{\boldsymbol{\omega}_b \times \mathbf{x}_{b,c}} + \mathbf{J}_{\text{CoM},j} \dot{\mathbf{q}}_j$$

$$\dot{\mathbf{x}}_c = \dot{\mathbf{x}}_b - [\mathbf{x}_{b,c}]_{\times} \boldsymbol{\omega}_b + \mathbf{J}_{\text{CoM},j} \dot{\mathbf{q}}_j$$



Average angular velocity

- We have a system of bodies so we have both base angular velocity and joint influence to the angular momentum

$$\mathbf{k}_G = \mathbf{I}_{\text{CoM}} \boldsymbol{\omega}_b + \mathbf{A}_{k,J} \dot{\mathbf{q}}_J$$

- The **average angular velocity** is then defined as the angular velocity that would generate the same angular momentum if the entire robot were considered instantaneously locked:

$$\mathbf{I}_{\text{CoM}} \bar{\boldsymbol{\omega}} = \mathbf{I}_{\text{CoM}} \boldsymbol{\omega}_b + \mathbf{A}_{k,J} \dot{\mathbf{q}}_J \quad \Rightarrow \quad \bar{\boldsymbol{\omega}} = \boldsymbol{\omega}_b + \mathbf{I}_{\text{CoM}}^{-1} \mathbf{A}_{k,J} \dot{\mathbf{q}}_J$$

- If we rewrite the dynamics equation using CoM velocity rather than base velocity, taking the following velocity transformation ${}_G T_B$

$$\underbrace{\begin{bmatrix} \dot{x}_{\text{com}} \\ \bar{\boldsymbol{\omega}} \\ \dot{\mathbf{q}}_J \end{bmatrix}}_{\dot{\mathbf{q}}_G} = \underbrace{3 \begin{bmatrix} I_{3 \times 3} & -[x_{b,\text{com}}] \times \\ 0 & I_{3 \times 3} \\ 0 & 0 \end{bmatrix}}_{{}_G T_B} \underbrace{\begin{bmatrix} J_{b,\text{com}}(q) \\ I_{\text{com}}^{-1} A_{k,j} \\ I_{n \times n} \end{bmatrix}}_{\text{Spatial transform that shifts twist from base to CoM}} \underbrace{\begin{bmatrix} \dot{x}_b \\ \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_J \end{bmatrix}}_{\dot{\mathbf{q}}_B}$$

$$\dot{\mathbf{q}}_B \in \mathbb{R}^{n+6}, \quad \dot{\mathbf{q}}_B \neq \dot{\mathbf{q}}_b \in \mathbb{R}^6.$$

State transformation for centroidal dynamics

- Velocity and acceleration transformations

$$\dot{q}_G = {}_G T_B \dot{q}_B,$$

$$\begin{aligned}\ddot{q}_G &= {}_G T_B \ddot{q}_B + {}_G \dot{T}_B \dot{q}_B \\ &= {}_G T_B \ddot{q}_B + {}_G \dot{T}_B {}_G T_B^{-1} \dot{q}_G.\end{aligned}$$

Solving for the
base
coordinates

$$\dot{q}_B = {}_G T_B^{-1} \dot{q}_G,$$

$$\ddot{q}_B = {}_G T_B^{-1} \left[\ddot{q}_G - {}_G \dot{T}_B {}_G T_B^{-1} \dot{q}_G \right]$$

- Jacobian and torque transformations (torque vector is invariant)

$$\dot{x}_c = J_B \dot{q}_B = J_G \dot{q}_G$$

$$\oplus$$

$$\dot{q}_G = {}_G T_B \dot{q}_B$$



$$J_B = J_G {}_G T_B$$



$$J_B^T = {}_G T_B^T J_G^T$$

$${}_G T_B^{-T} \begin{bmatrix} 0 \\ \tau \end{bmatrix} = \begin{bmatrix} 0 \\ \tau \end{bmatrix}$$

- Dynamics after the change of generalized velocity ($T = {}_G T_B$)

$$M_B T^{-1} \ddot{q}_G - M_B T^{-1} \dot{T} T^{-1} \dot{q}_G + C_B T^{-1} \dot{q}_G + g_B = \begin{bmatrix} 0 \\ \tau \end{bmatrix} + T^T J_G^T f.$$

- Left multiplication by T^{-T} produces

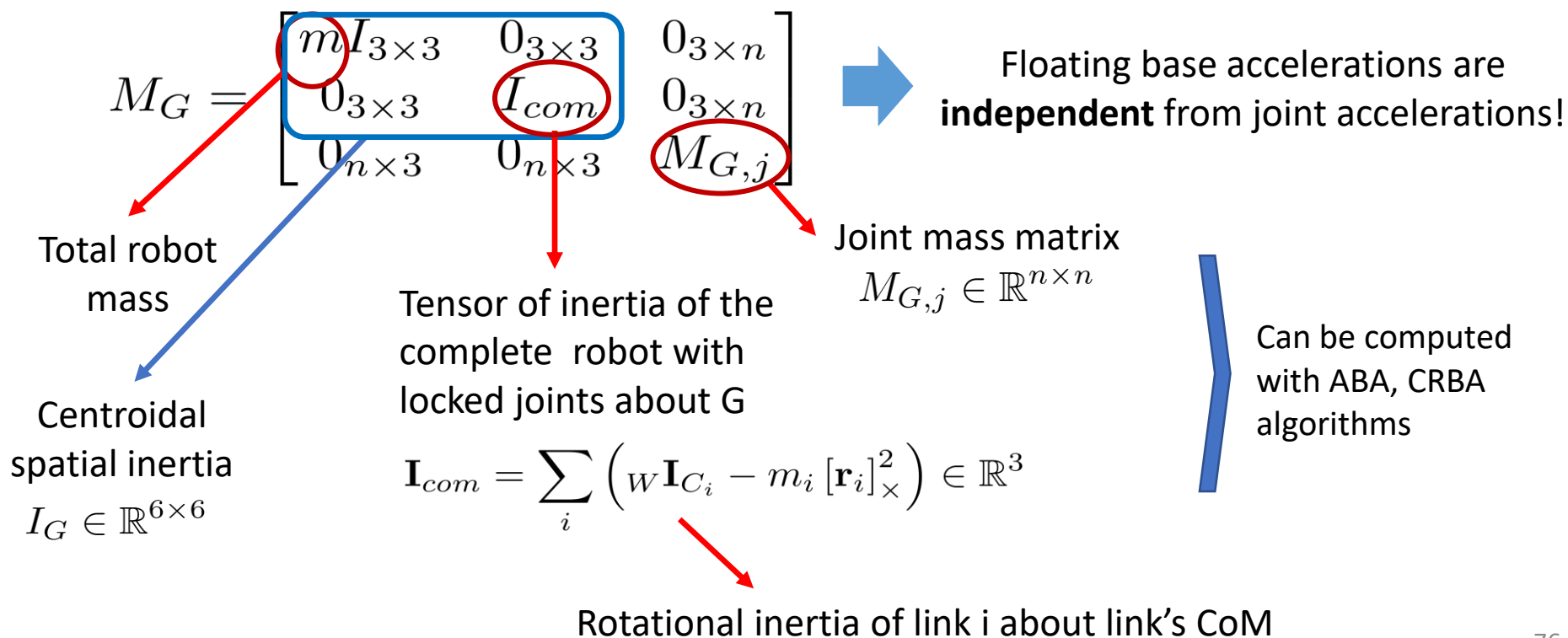
$$\underbrace{T^{-T} M_B T^{-1}}_{M_G} \ddot{q}_G + \underbrace{\left[-T^{-T} M_B T^{-1} \dot{T} T^{-1} + T^{-T} C_B T^{-1} \right]}_{C_G} \dot{q}_G + \underbrace{T^{-T} g_B}_{g_G} = S^T \tau + J_{G,c}^T f.$$

State transformation for centroidal dynamics

- We obtain the dynamics when the floating base frame is attached to the CoM

$$M_G \ddot{q}_G + \underbrace{C_G}_{c_G} \dot{q} + g_G = S^T \tau + J_{c,G}^T f.$$

- Fact 7:** In this case the inertia matrix has full **block-diagonal** structure!



State transformation for centroidal dynamics

- **Fact 8:** Gravity term is constant and influence only the acceleration of the CoM

$$g_G = T^{-T} g_B = \begin{bmatrix} 0 \\ 0 \\ mg \\ 0_{3 \times 1} \\ 0_{m \times 1} \end{bmatrix}. \quad \rightarrow \text{Independent from } q!$$

- The Coriolis matrix is partitioned as

$$C_G = \begin{bmatrix} 0_{3 \times 3} & 0_{3 \times (3+n)} \\ 0_{(3+n) \times 3} & \bar{C} \end{bmatrix} = \begin{bmatrix} 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times n} \\ 0_{3 \times 3} & \# & \# \\ 0_{n \times 3} & \# & \# \end{bmatrix} \in \mathbb{R}^{(n+6) \times (n+6)}.$$

- The Coriolis vector hence becomes: $c_G = C_G(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \begin{bmatrix} 0_{3 \times 3} \\ \# \\ \# \end{bmatrix} \in \mathbb{R}^{n+6}$

- For three contacts, the transpose of the base contact Jacobian is shown as

$$J_{G,c,b}^T = \begin{bmatrix} I_{3 \times 3} & I_{3 \times 3} & I_{3 \times 3} \\ [x_{f_1} - x_c]_{\times} & [x_{f_2} - x_c]_{\times} & [x_{f_3} - x_c]_{\times} \end{bmatrix}.$$

Sparsity of floating-base dynamics expressed at CoM

- Due to **Facts 7-8** the **underactuated** and **actuated** parts can be fully decoupled!

$$\begin{array}{c} 3 \\ 3 \\ n \end{array} \begin{bmatrix} m\mathbf{I}_{3 \times 3} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{com} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{M}_{G,j} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{x}}_c \\ \dot{\bar{\omega}} \\ \ddot{\mathbf{q}}_j \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \# \\ \# \end{bmatrix} + \begin{bmatrix} m\mathbf{g} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \boldsymbol{\tau} \end{bmatrix} + \begin{bmatrix} \mathbf{J}_{G,c,b}^T \\ \mathbf{J}_{G,c,j}^T \end{bmatrix} \mathbf{f}.$$

- Fact 9:** The first six rows of the underactuated dynamics correspond to the **Newton-Euler equations** of the whole system, i.e., the robot simplified as a single rigid body with **variable inertia**.

$$\begin{cases} m\ddot{\mathbf{x}}_c \\ I_{com}\dot{\bar{\omega}} + \bar{\omega} \times I_{com}\bar{\omega} \end{cases} = \begin{cases} \sum_{i=1}^c f_i + mg, & \text{Newton equation} \\ \sum_{i=1}^c (x_{f,i} - x_c) \times f_i. & \text{Euler equation} \end{cases}$$

- Newton-Euler equations can be obtained by balancing forces/moment acting on the robot as a whole

Newton-Euler equations

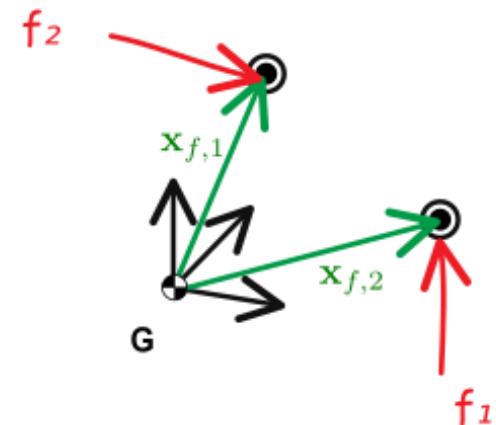
- Describe the motion of a rigid body with forces /moments applied at CoM
- They are derived from the conservation of linear/angular momentum
- Newton equation:** the variation of linear momentum is written as

$$\frac{d}{dt}(\ell) = \frac{d}{dt}(m\dot{x}_{com}) = F_{ext}, \longrightarrow m\ddot{x}_{com} = \sum_i f_i + mg \quad \textbf{Linear dynamics}$$

- Euler equation:** The variation of angular momentum (about point G) is written as

$$\dot{k}_G = \sum_{i=1}^c \underbrace{(x_{f_i} - x_{com}) \times f_i}_{\text{External moments balance about G}} = M_{ext}.$$

Application point of force f_i



- We choose the CoM (G) as the pole. We express all vectors in the world frame.

Euler equation

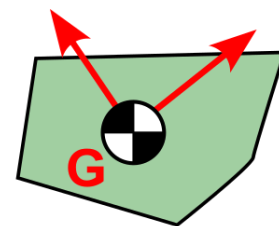
- The change in angular momentum is due to external moments

$$\dot{k}_G = \frac{d}{dt} ({}_W \mathbf{I}_{com} \boldsymbol{\omega}) = {}_W M_{ext}$$

$$= {}_W \mathbf{I}_{com} \dot{\boldsymbol{\omega}} + \frac{d}{dt} ({}_W \mathbf{R}_G {}_G \mathbf{I}_{com} {}_W \mathbf{R}_G^T) \boldsymbol{\omega},$$

Differentiation rule

${}_W \dot{\mathbf{I}}_{com} \neq 0$ In an inertial frame (even if attached at G) is time varying



Robot inertia expressed in a rotating frame is constant

$$\begin{aligned} \dot{\mathbf{R}} &= [\boldsymbol{\omega}]_{\times} \mathbf{R}, \\ \dot{\mathbf{R}}^T &= \mathbf{R}^T [\boldsymbol{\omega}]_{\times}^T \\ &= -\mathbf{R}^T [\boldsymbol{\omega}]_{\times}. \end{aligned}$$

$$\mathbf{A}^T = -\mathbf{A}$$

$$= {}_W \mathbf{I}_{com} \dot{\boldsymbol{\omega}} + \left({}_W \dot{\mathbf{R}}_G {}_G \mathbf{I}_{com} {}_W \mathbf{R}_G^T + {}_W \mathbf{R}_G {}_G \mathbf{I}_{com} {}_W \dot{\mathbf{R}}_G^T \right) \boldsymbol{\omega}$$

$$= {}_W \mathbf{I}_{com} \dot{\boldsymbol{\omega}} + [\boldsymbol{\omega}]_{\times} \underbrace{{}_W \mathbf{R}_G {}_G \mathbf{I}_{com} {}_W \mathbf{R}_G^T}_{{}_W \mathbf{I}_{com}} \boldsymbol{\omega} - {}_W \mathbf{R}_G {}_G \mathbf{I}_{com} {}_W \mathbf{R}_G^T [\boldsymbol{\omega}]_{\times} \boldsymbol{\omega}.$$

$$= {}_W \mathbf{I}_{com} \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times ({}_W \mathbf{I}_{com} \boldsymbol{\omega}) - {}_W \mathbf{I}_{com} \underbrace{\boldsymbol{\omega} \times \boldsymbol{\omega}}_0.$$

[Murray sec. 2.4]

Angular momentum conservation

- In absence of external forces the momentum conserves (principle of dynamics)

$$\begin{cases} m\dot{\mathbf{x}}_c = \text{const} \\ \mathbf{I}_{com}\omega = \text{const} \end{cases} \rightarrow \text{For a rigid body, zero angular momentum implies zero angular velocity and the orientation remains **constant**}$$

- But...we have an articulated robot not a rigid body! $I_{com}(q)$
- If we differentiate we see there are 2 contributions to the rate of change of the angular momentum

$$0 = I_{com}(q)\dot{\omega} + \dot{I}_{com}(q)\omega = \dot{k}$$

Angular acceleration of the whole robot

Inertia variation due to Internal movements of the joints (gesticulation)



- It is possible to reorient the robot moving the legs! (e.g. like a falling cat or a dancer)

- even when the total angular momentum is zero, moving the joints can generate a nonzero base angular velocity

$$\dot{\omega} = -I_{com}(q)^{-1}\dot{I}_{com}(q)\omega$$

- The base orientation will evolve according to $\dot{\mathbf{R}}_b = [\omega_b]_{\times} \mathbf{R}_b$

Centroidal momentum matrix

[Orin et al. AURO, 213]

- The centroidal momentum can be also expressed in terms of generalized coordinates

$$\mathbf{h}_G = \begin{bmatrix} \mathbf{l}_G \\ \mathbf{k}_G \end{bmatrix} = \mathbf{A}_G(\mathbf{q})\dot{\mathbf{q}}_B, \quad \text{where } \mathbf{A}_G \text{ is the **centroidal momentum matrix (CMM)**}.$$

- The output is on centroidal coordinates but we use base coordinates!
- Partitioning it according to base and joint velocities gives

$$\mathbf{h}_G = \mathbf{A}_{G,b}\dot{\mathbf{q}}_b + \mathbf{A}_{G,j}\dot{\mathbf{q}}_j \quad \text{With} \quad \mathbf{A}_G = \begin{bmatrix} \mathbf{A}_{G,b} & \mathbf{A}_{G,j} \end{bmatrix} \in \mathbb{R}^{6 \times (6+n)}$$

$\oplus$



 locked centroidal spatial inertia

$$\dot{\mathbf{q}}_B = \begin{bmatrix} \dot{\mathbf{q}}_b \in \mathbb{R}^6 \\ \dot{\mathbf{q}}_j \in \mathbb{R}^n \end{bmatrix} \quad \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{\text{com}}(\mathbf{q}) \end{bmatrix} \in \mathbb{R}^{6 \times 6}$$

- However the centroidal momentum is also expressed in terms of average angular velocity

$$\mathbf{h}_G = \begin{bmatrix} m\dot{\mathbf{x}}_{\text{com}} \\ \mathbf{I}_{\text{com}}(\mathbf{q})\overline{\boldsymbol{\omega}} \end{bmatrix} \quad \text{differentiating} \quad \dot{\mathbf{h}}_G = \begin{bmatrix} m\ddot{\mathbf{x}}_{\text{com}} \\ \mathbf{I}_{\text{com}}\dot{\overline{\boldsymbol{\omega}}} + \overline{\boldsymbol{\omega}} \times \mathbf{I}_{\text{com}}\overline{\boldsymbol{\omega}} \end{bmatrix}$$

Centroidal momentum matrix

- From Newton-Euler equations we have seen that this equals the resultant external wrench about the CoM:

$$\dot{\mathbf{h}}_G = \begin{bmatrix} m\mathbf{g} + \sum_i \mathbf{f}_i \\ \sum_i (\mathbf{p}_{f,i} - \mathbf{x}_{\text{com}}) \times \mathbf{f}_i + \sum_i \tau_i \end{bmatrix} = \mathbf{w}_G^{ext}$$

Only for humanoids

- Similarly, the CMM is a tool that allows us to use base generalized coordinates to deal with the rate of change of centroidal momentum

$$\dot{\mathbf{h}}_G = \mathbf{A}_G(\mathbf{q})\ddot{\mathbf{q}}_B + \dot{\mathbf{A}}_G(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}_B = \mathbf{w}_G^{ext}$$

- CMM can be computed by many kinematics libraries (e.g. Pinocchio)

```
pin.computeCentroidalMap(model, data, q)  
Ag = data.Ag
```

References

- Average Angular Velocity, H. Essen, 1992.
- On Centroidal Dynamics and Integrability of Average Angular Velocity, A. Saccon et al., 2017.
- A Unified View of the Equations of Motion used for Control Design of Humanoid Robots, S. Traversaro, 2017.
- Centroidal Momentum Matrix of a Humanoid Robot: Structure and Properties, D.Orin et al., 2008.
- Robust multi-contact dynamical motion planning using contact wrench set, H. Dai PhD Thesis, 2016.
- A Mathematical Introduction to Robotic Manipulation - Richard M. Murray, Zexiang Li, S. Shankar Sastry. (section 2.4)
- MIT1607_F09 - Lecture L28 - 3D Rigid Body Dynamics: Equations of Motion, Euler's Equations, J. Peraire.
- Spacecraft Attitude Dynamics and Control, Dispense del Corso di Dinamica e Controllo di Assetto di Satelliti, G. Avanzini 2008.

Take home messages

Legged robots are difficult to control:

- Underactuation | control CoM only through grfs
- Discontinuous dynamics | swing, stance, flight phase
- Actuation limits | must carry his own weight

A controller that accounts for this complex dynamics and ground interactions is crucial to **balance** and at the same time to avoid **slippage**

Outline

Brief History of legged Robots

Floating Base Robots

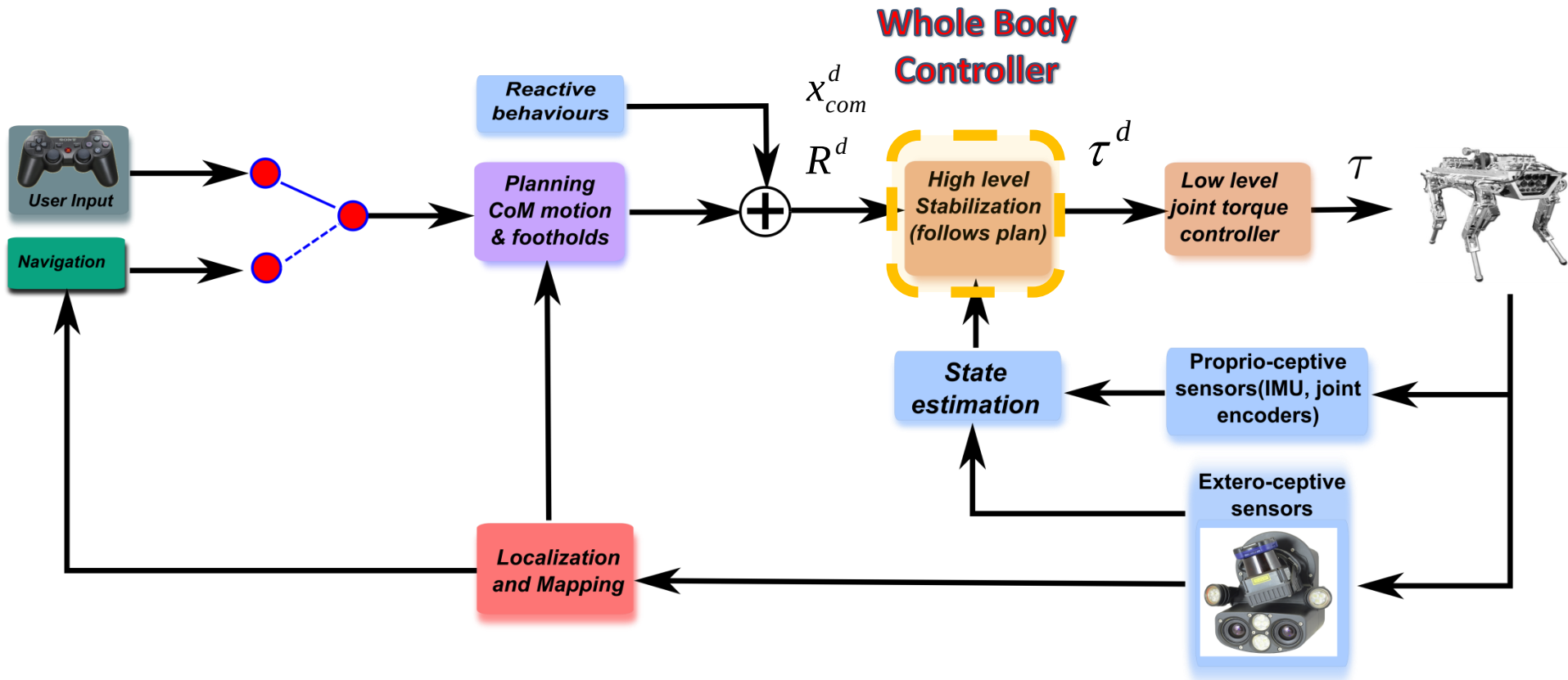
Basic Concepts in Legged Locomotion

Modeling Legged robots

Towards Newton-Euler equations

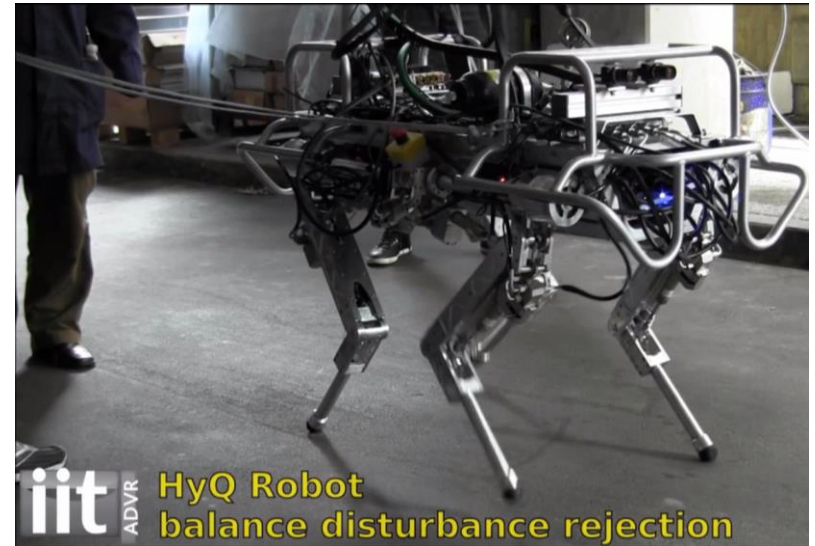
Quasi-static Trunk Controller

Locomotion Framework

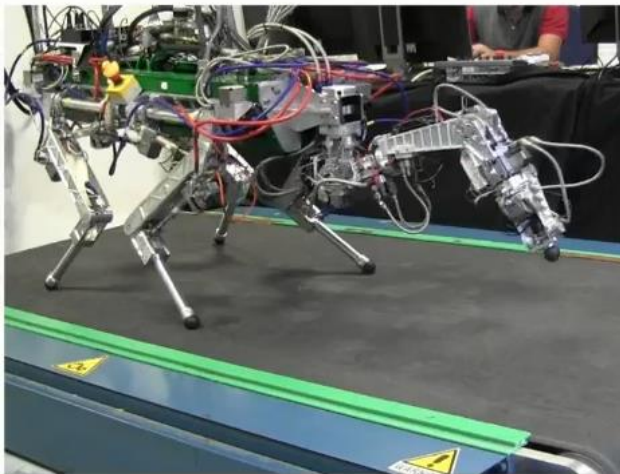


Trunk Controller

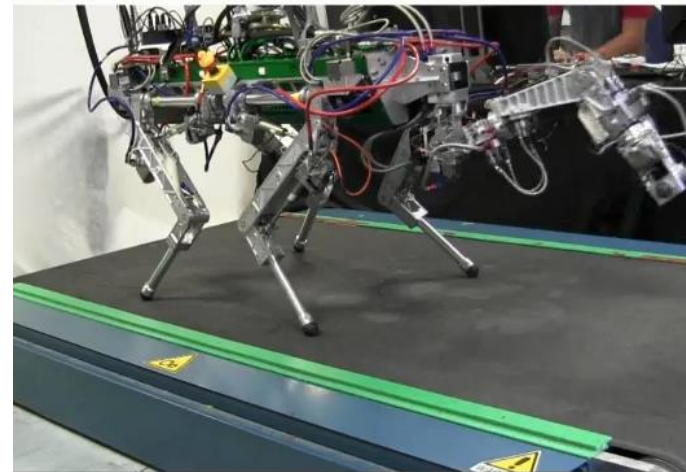
- It computes joint torques:
 - ① to stabilize trunk orientation
 - ② to stabilize CoM position



STABILIZATION OFF

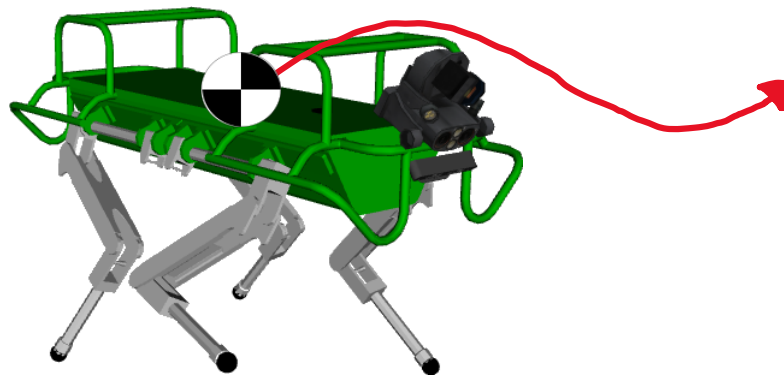


STABILIZATION ON



Projection-based Quasi-static controller

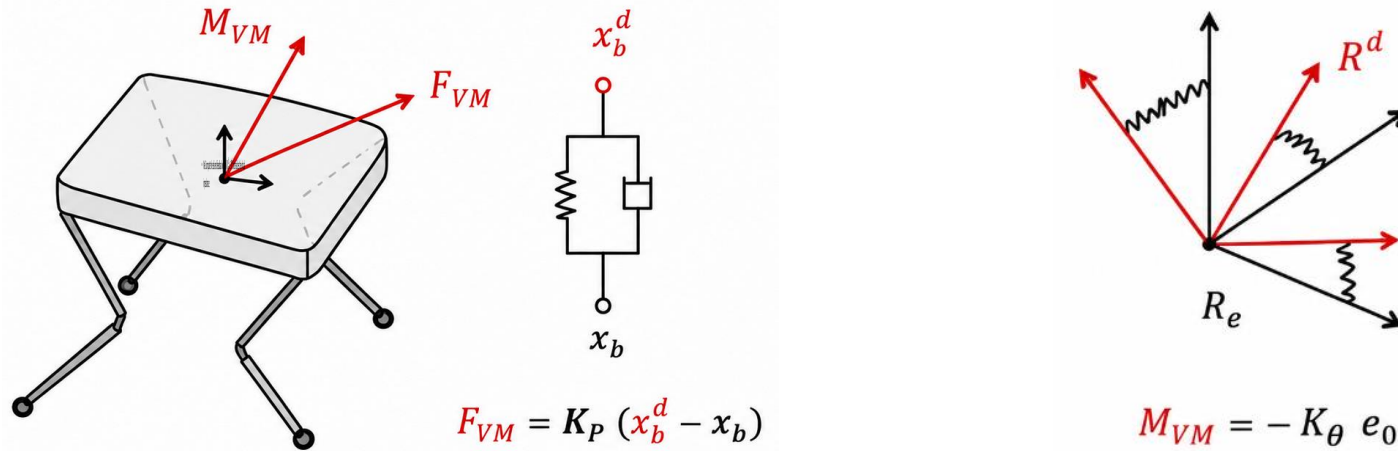
- Quasi-static assumption: neglects accelerations/velocities
- Computes contact forces/torques to produce the same effect of **virtual** forces and moments at the CoM (e.g. emulate virtual springs/dampers attached to the CoM)
- These virtual elements track a reference trajectory coming from a planner



- Applicability: slow motions, quasi static motions
- Also called **virtual model approach**, introduced by J.Pratt in 1997

Desired wrench computation: virtual impedance

- First we design a virtual impedance to track references for the CoM frame position and orientation.



- We Compute the desired feedback wrench $W_{fbk}^d \in \mathbb{R}^6$ that realizes the virtual impedance:

$$W_{fbk}^d = \begin{bmatrix} W_{lin} \\ W_{ang} \end{bmatrix} \in \mathbb{R}^6 \quad \begin{cases} W_{lin}^d = K_p(x_b^d - x_b) + D_p(\dot{x}_b^d - \dot{x}) \\ W_{ang}^d = -K_\theta e_o + D_\theta(\dot{\omega}_b^d - \omega) \end{cases} \quad \text{where } e_o \text{ is the orientation error.}$$

- Add a gravity compensation and a feed-forward term to improve tracking

$$W^d = W_{fbk} + W_g + W_{ffwd} \quad W_g = \begin{bmatrix} mg \\ \mathbf{0}_{3 \times 1} \end{bmatrix} \quad W_{ffwd} = \begin{bmatrix} m\ddot{\mathbf{x}}_{com}^d \\ \mathbf{I}_{com}\dot{\boldsymbol{\omega}}^d \end{bmatrix}$$

Mapping to torques

- Exploiting dynamics decoupling (**Fact 9**) and neglecting accelerations/velocities (quasi-static assumption) from the **underactuated** equation we can compute the contact forces that realize the desired wrench W^d

$$\begin{bmatrix} m\mathbf{I}_{3 \times 3} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{com} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{x}}_{com} \\ \dot{\boldsymbol{\omega}} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \# \end{bmatrix} + \begin{bmatrix} m\mathbf{g} \\ \mathbf{0} \end{bmatrix} = {}_G\mathbf{J}_{G,c,b}^T \mathbf{f}. \Rightarrow \boxed{\mathbf{f}^d = (J_{G,c,b}^T)^\dagger W^d}$$

- from the **actuated** equation map contact forces into joint torques

$$\cancel{{}_G\mathbf{M}_j \ddot{\mathbf{q}}_j} + \cancel{\mathbf{h}_j} = \boldsymbol{\tau} + {}_G\mathbf{J}_{c,j}^T \mathbf{f}. \Rightarrow \boxed{\boldsymbol{\tau}^d = -J_{G,c,j}^T \mathbf{f}^d = -J_{G,c,j}^T (J_{G,c,b}^T)^\dagger W^d}$$

- where

$$J_{G,c,b}^T = \begin{bmatrix} I_{3 \times 3} & \dots & I_{3 \times 3} \\ [x_{f_1} - x_{com}]_{\times} & \dots & [x_{f_c} - x_{com}]_{\times} \end{bmatrix}$$

- Extends the concept of virtual works considering the mapping of a wrench at CoM into joint torques
- Exploits the $J_{c,b}^T$ matrix for the mapping between CoM wrench and contact forces
- Remark: $\mathbf{f}$ are the contact forces of the legs in **stance**, therefore both $\mathbf{f}$ and $J_{c,b}^T$ change dimension when the contact state changes

Quasi-static controller pros & cons



Very simple



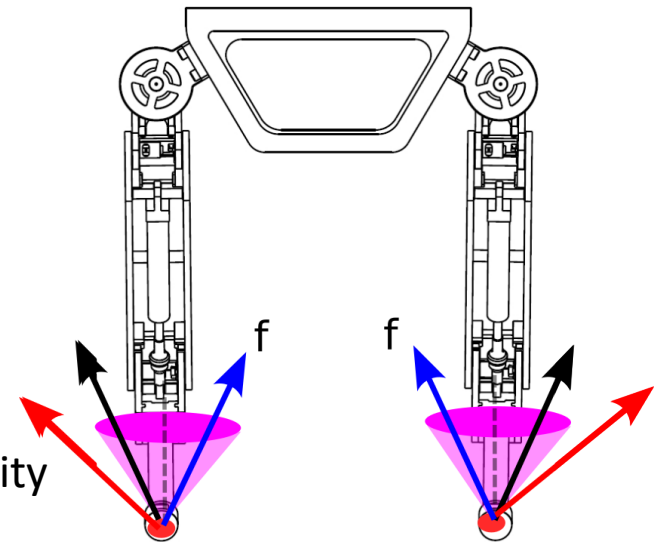
We can exploit force redundancy getting different solutions by setting a weighting matrix in the pseudoinverse

$$\tau^d = -J_{cj}^T \mathbf{f}^d = -J_{cj}^T \underbrace{(J_b^T)^\#}_{\text{Weighted pseudo-inverse}} \mathbf{W} W^d$$

Weighted pseudo-inverse



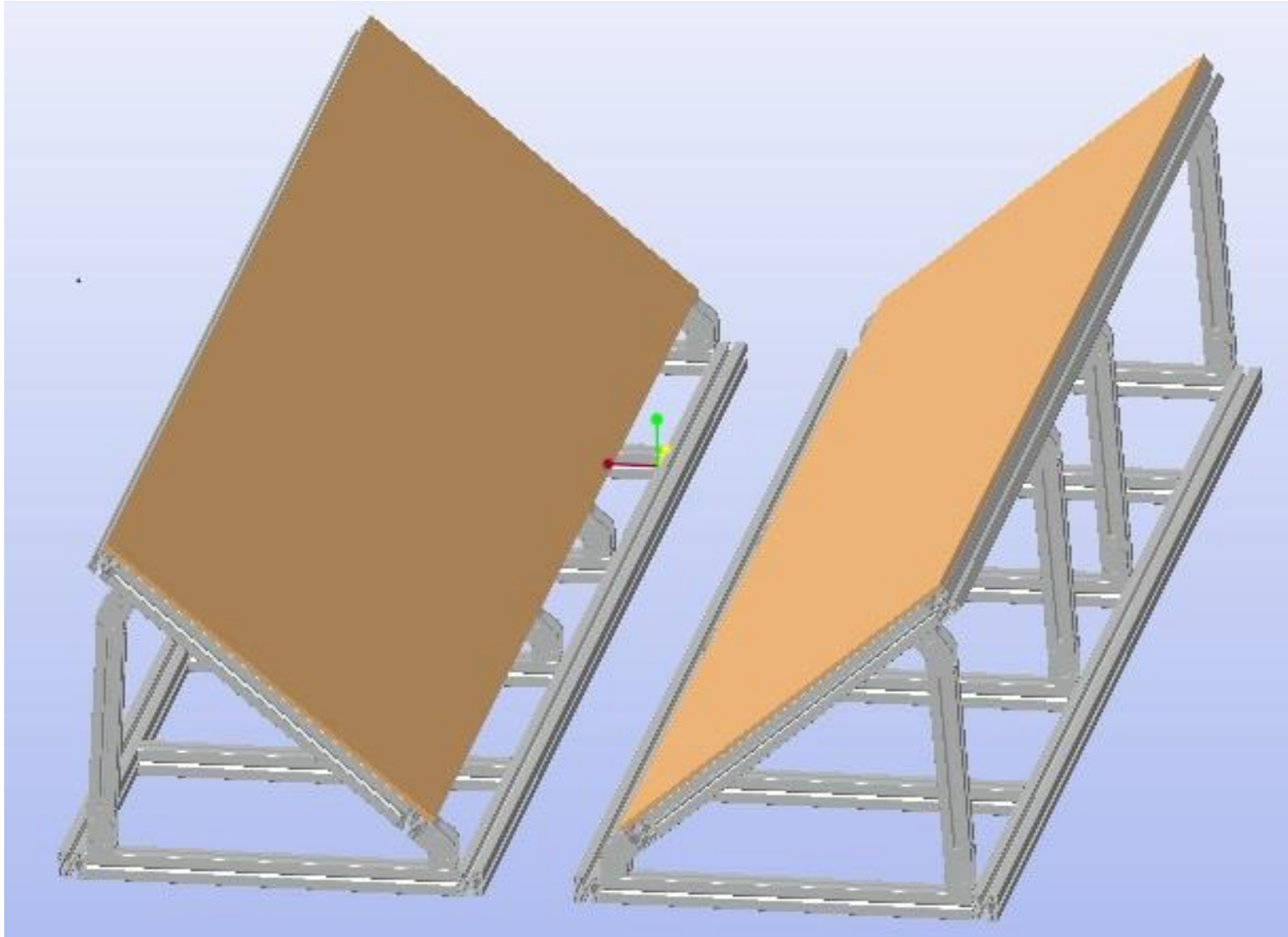
No guarantee on satisfaction of contact unilaterality (I can have $f_z < 0$) or slippage constraints



To incorporate them we need to cast the control problem as an optimization problem (QP) see the lecture of A. Del Prete!

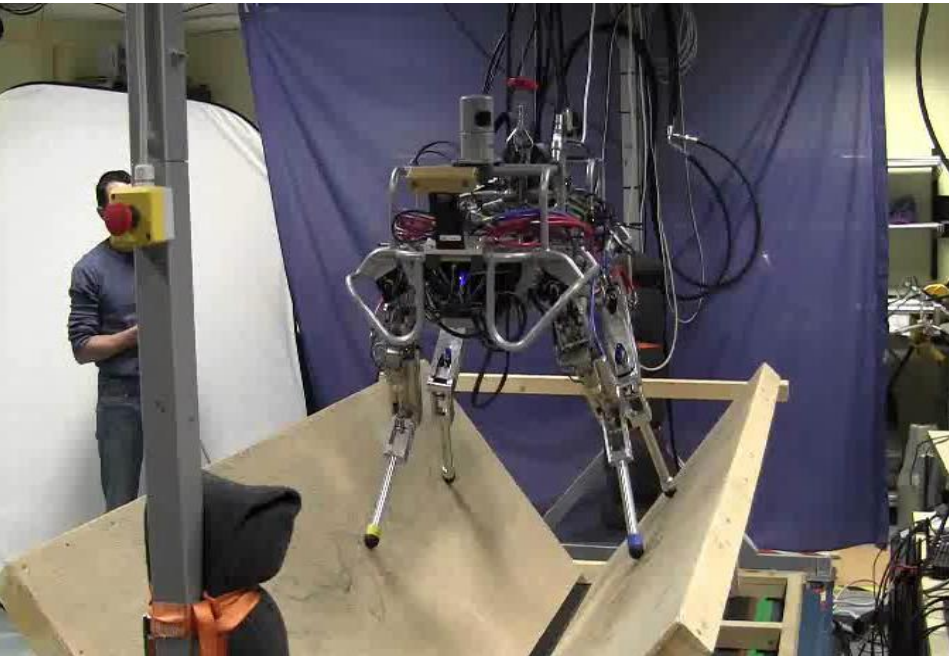
Optimization of contact forces: testrig

V-shaped wall

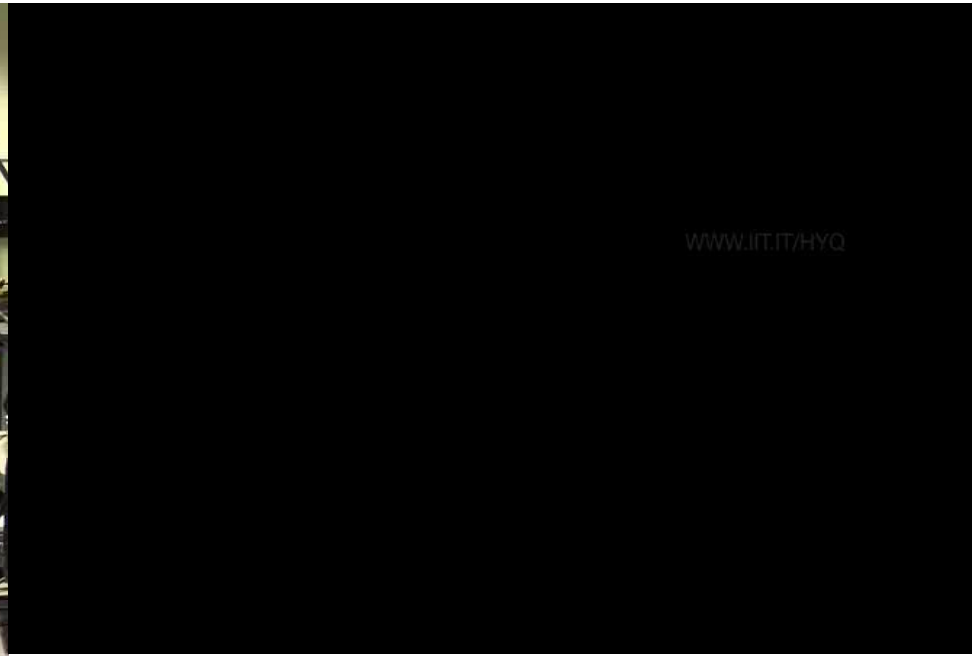


Optimization of contact forces

OPTIMIZATION OFF

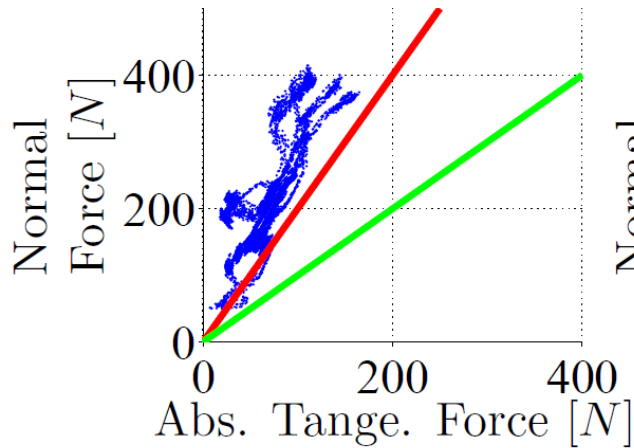


OPTIMIZATION ON

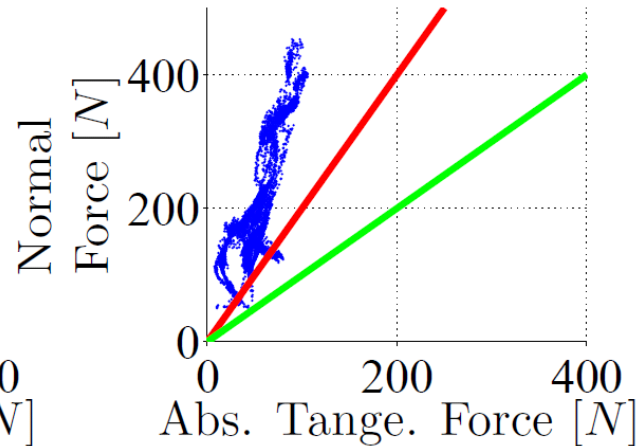
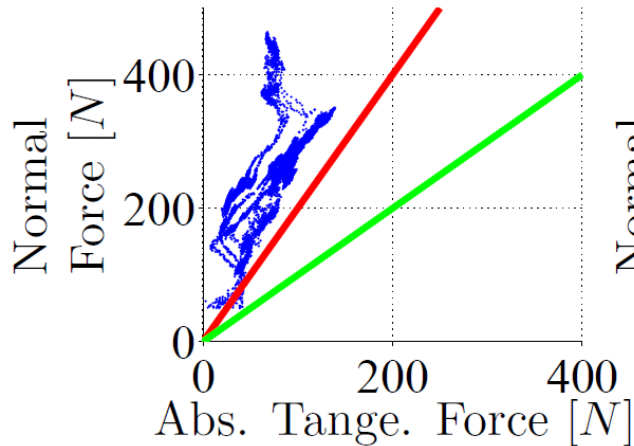
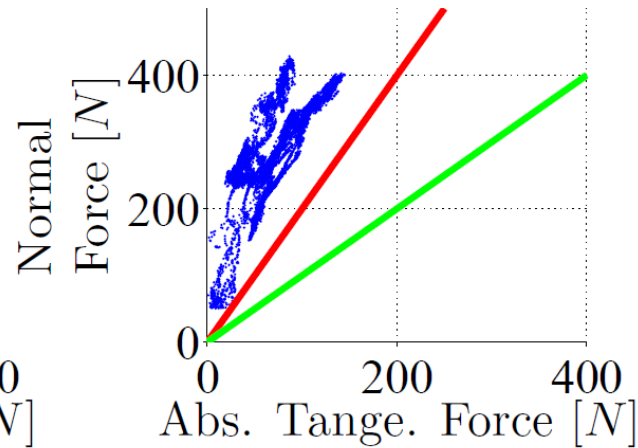


Experimental results: friction cones

Controller $\mu = 0.5$



Estimated $\mu = 1.0$



- **Conservative** friction coefficient ensures better tolerance to **uncertainties** in the estimation of the **normal** of the surface
- tolerance to slope estimation errors of up to 18°